

Apuntes clase 4

Ejecución Directa Limitada

19/02/2026

user space vs. kernel space

JULIA EVANS
@b0rk

drawings.jvns.ca

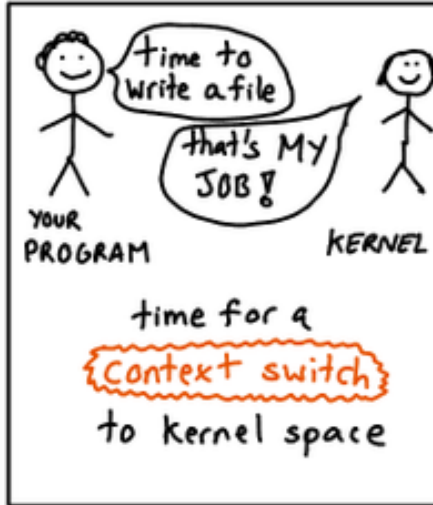
the Linux kernel has
millions of lines of code

- ✱ read+write files
- ✱ decide which programs get to use the CPU
- ✱ make the keyboard work

When Linux kernel code runs, that's called

kernel space

When your program runs, that's
user space



your program switches
back and forth

```
str = "my string"
```

```
x = x + 2
```

```
file.write(str) ← ✱ switch to kernel space ✱
```

```
y = x + 4
```

```
str = str * y ← ✱ and we're back to user space! ✱
```

timing your process

\$ time find /home

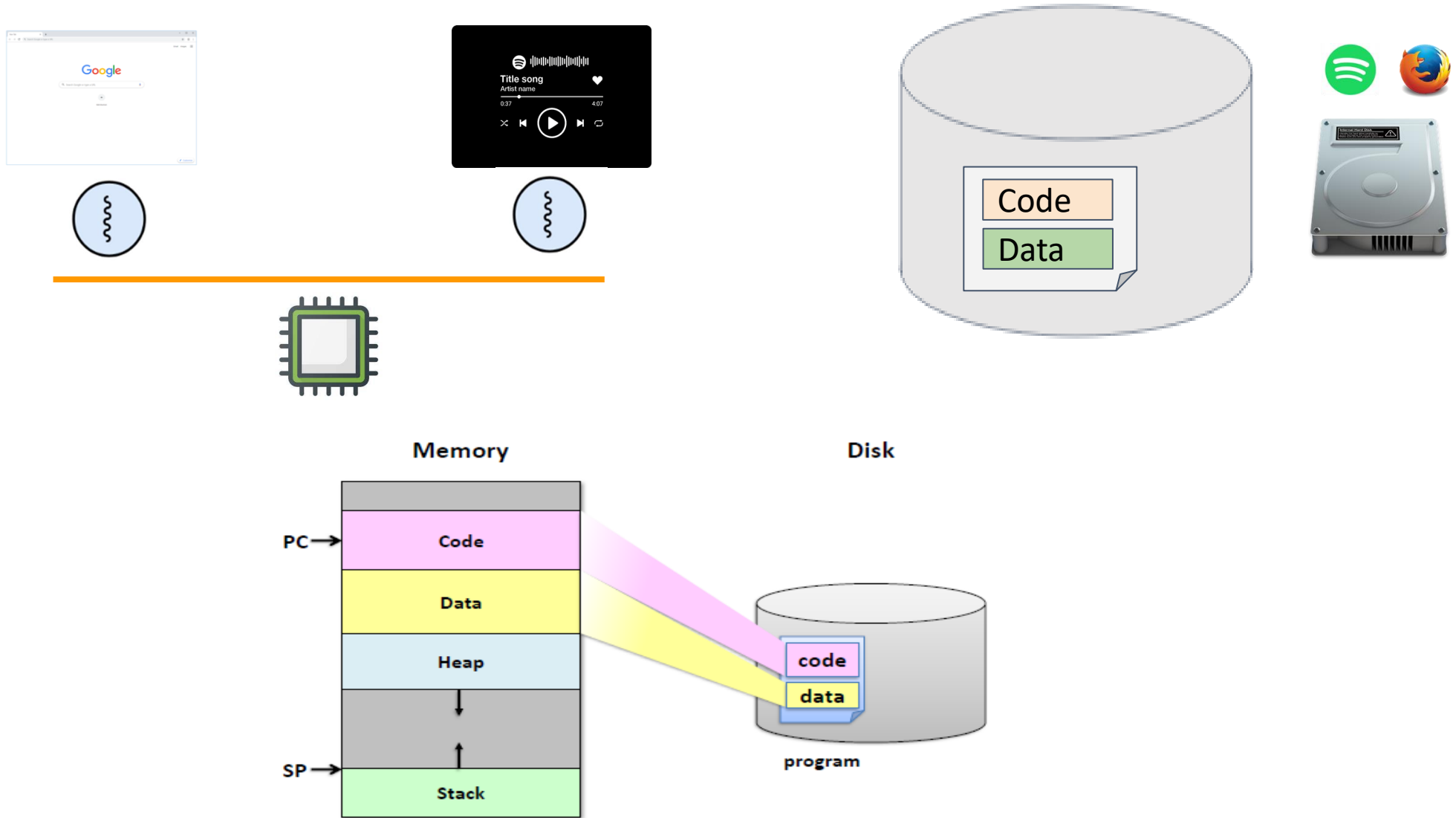
0.15 user 0.73 system

↑
time spent in
your process

↑
time spent by
the kernel doing
work for your
process

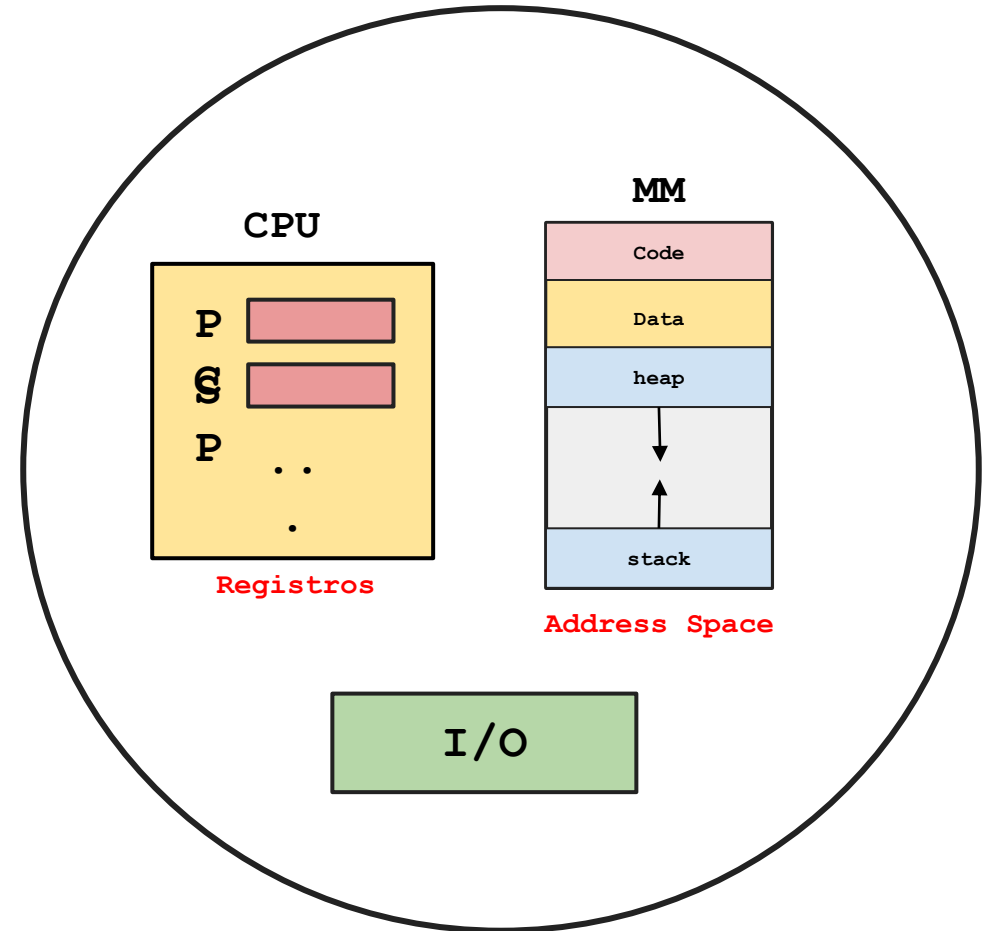
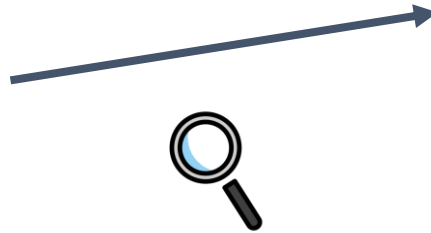
Url: <https://twitter.com/b0rk/status/804200666226900992>

Programa .vs. Proceso



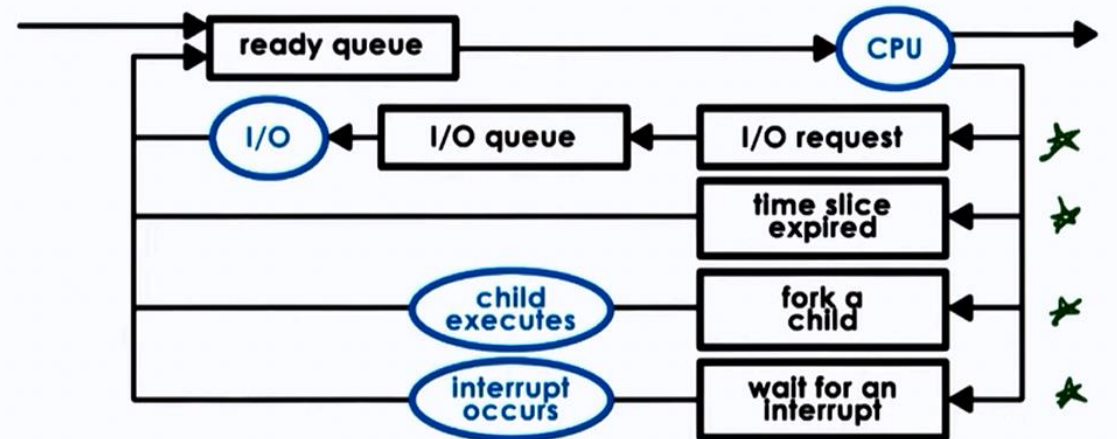
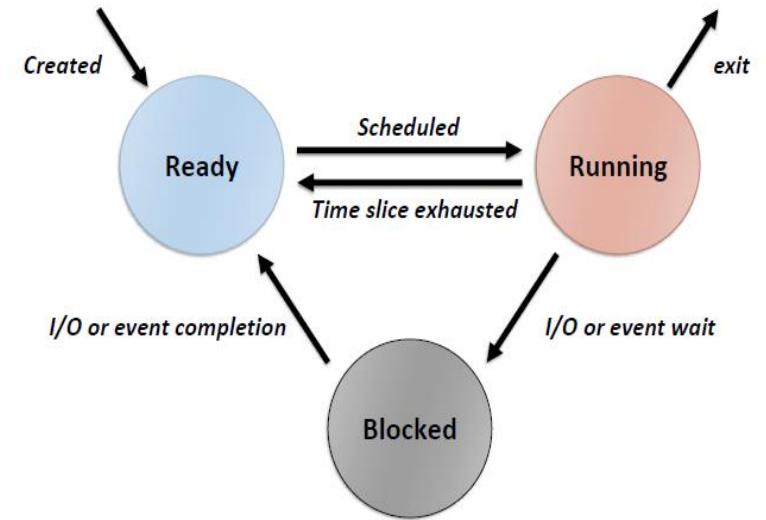
El proceso como abstracción

Proceso = CPU + Memory + I/O Info



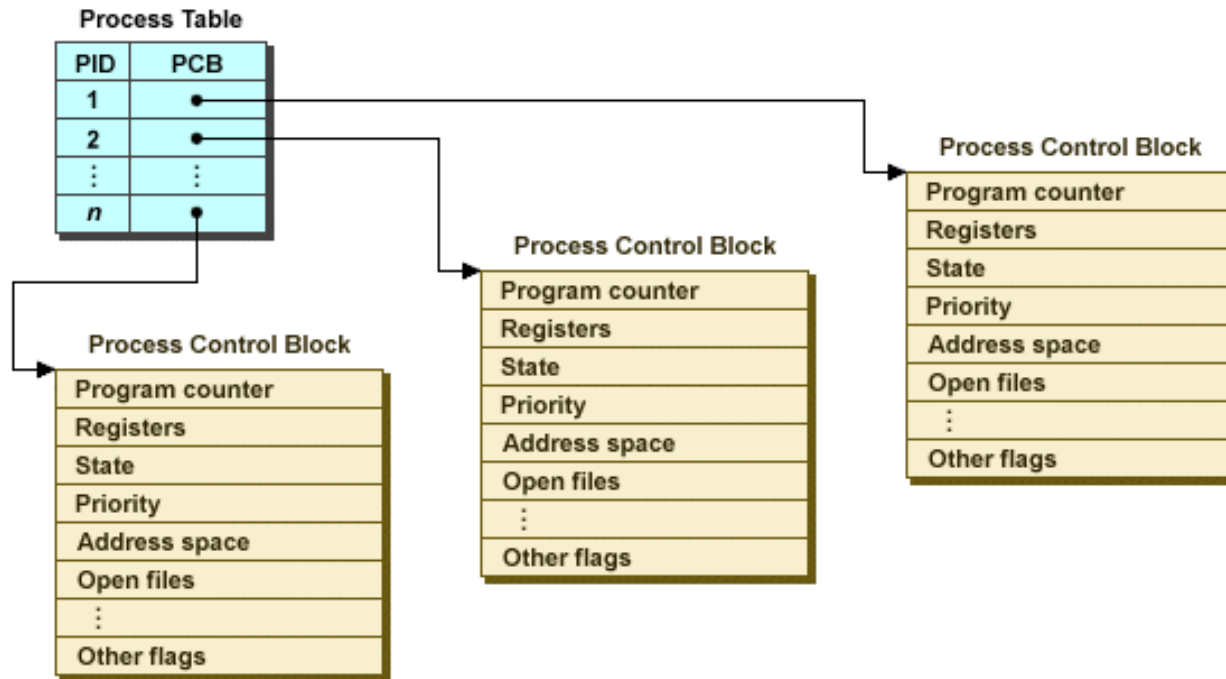
API de procesos

Función	Descripción
Crear (create)	Crea un nuevo proceso para ejecutar un programa.
Eliminar (destroy)	Forzar la detención de un programa en ejecución.
Esperar (wait)	Espera que un proceso termine su ejecución.
Operaciones de control	Ej. Suspender un proceso.
Estado (status)	Información de estado del proceso.



Estructuras de datos asociadas a un proceso

- Process table (PT)
- Process control Block (PCB)



Estructura proc del xv6

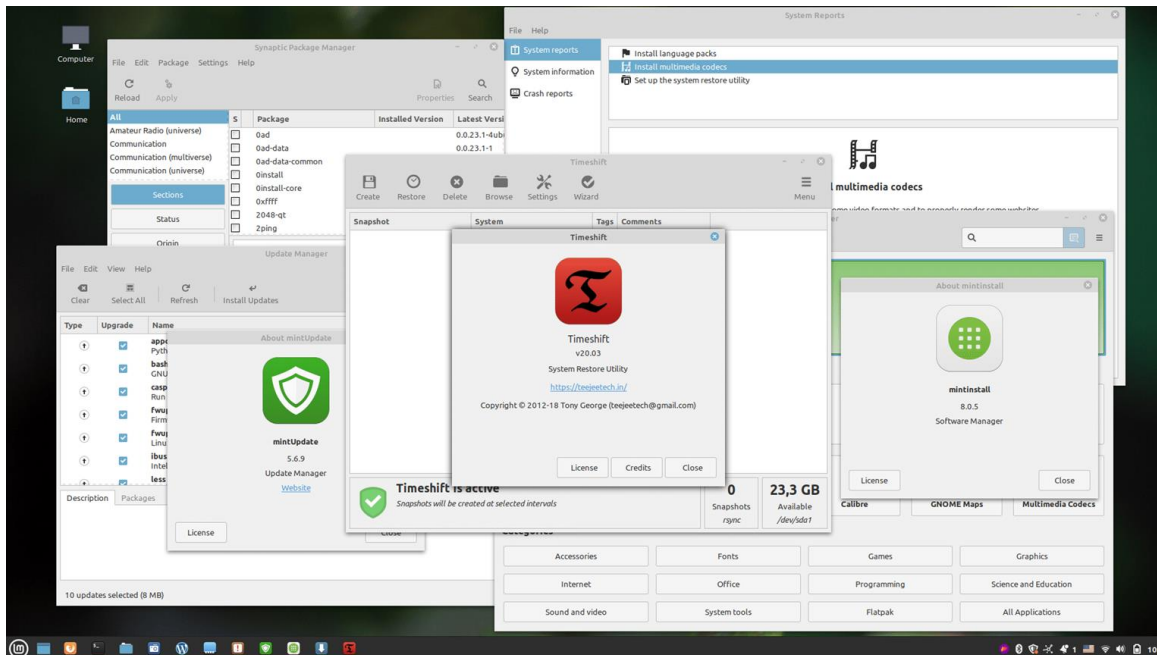
```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
    int eip;
    int esp;
    ...
    int edi;
    int ebp;
};

// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };

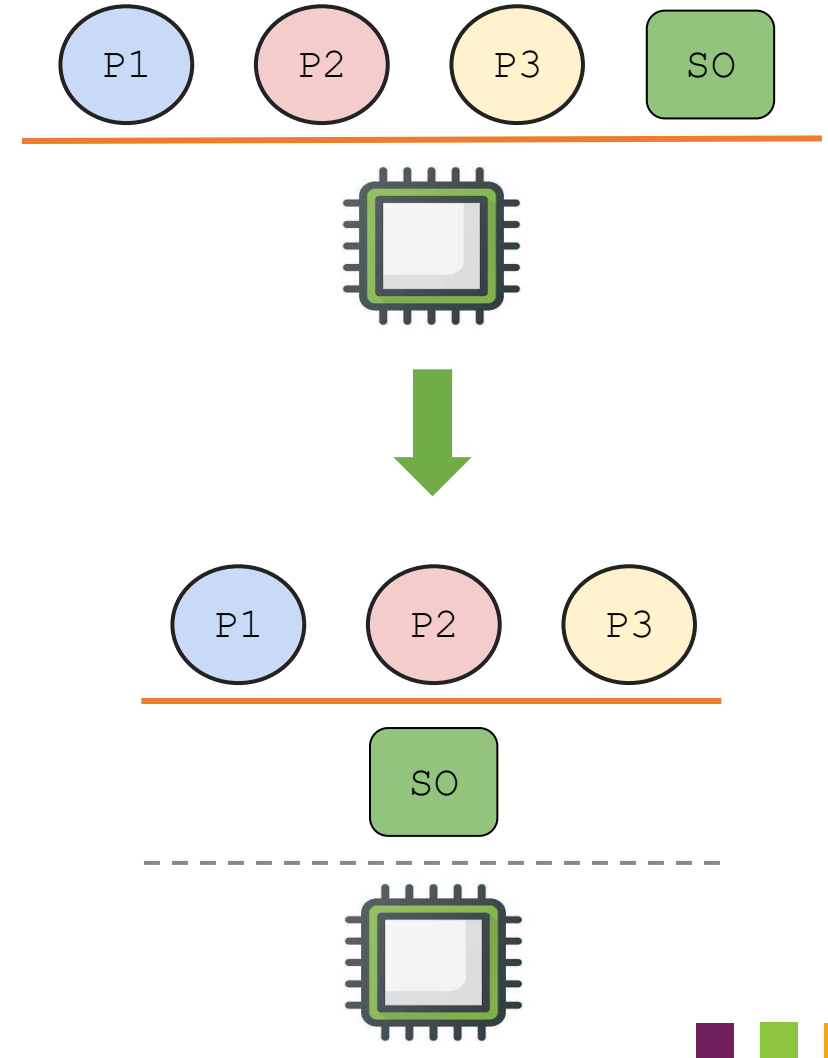
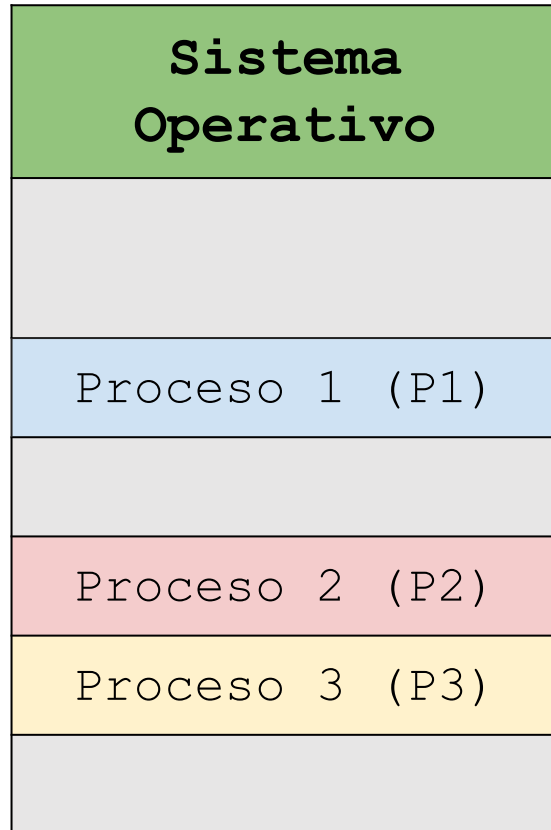
// the information xv6 tracks about each process
// including its register context and state
struct proc {
    char *mem;
    char *kstack;
    enum proc_state state;
    int pid;
    ...
    interrupt
};
```

Multiprogramación

¿Cómo se logra la virtualización de la CPU de modo que cada proceso crea que tiene su propia CPU?

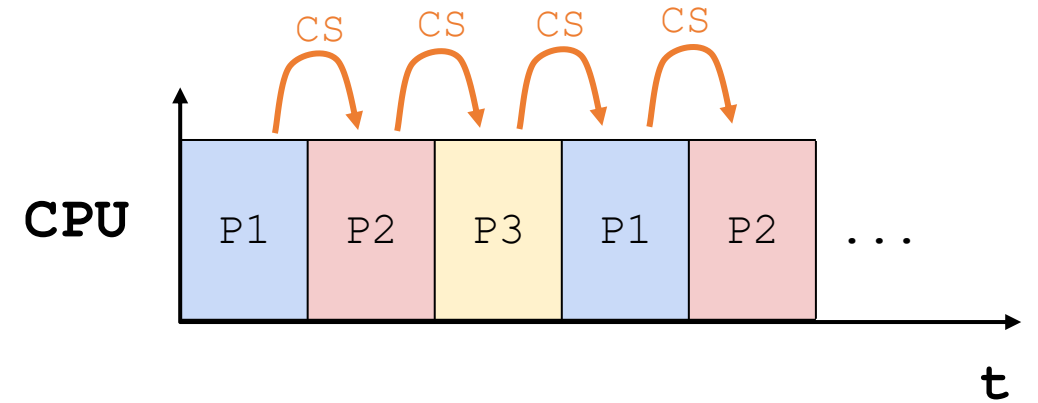
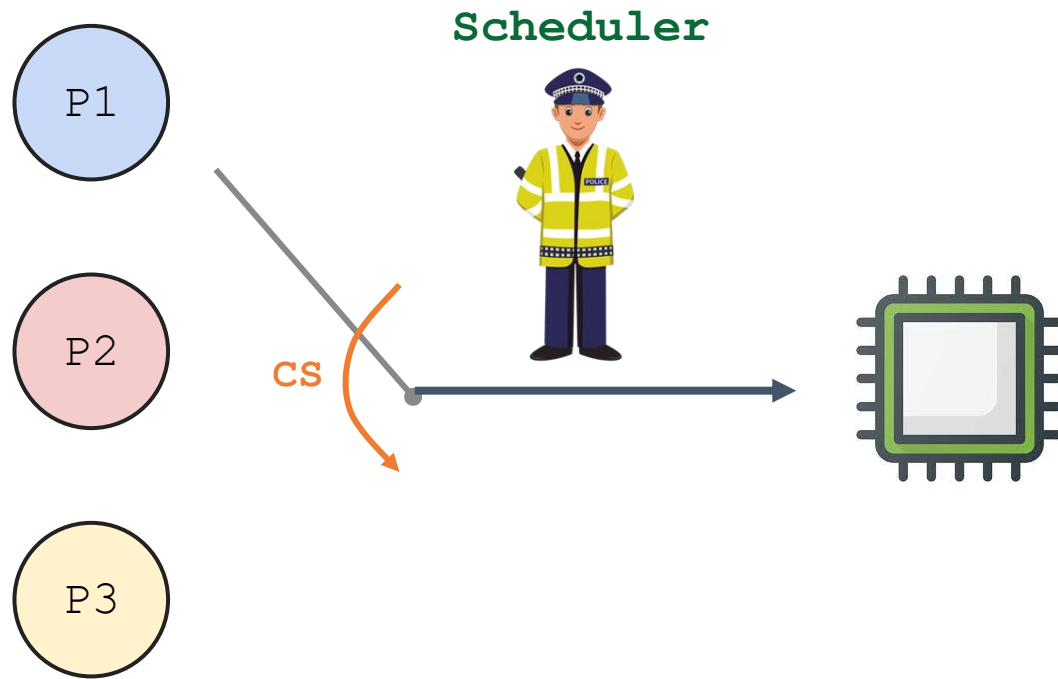


Multiprogramación



Multiprogramación

Time Sharing



Ejecución directa limitada (EDL)

Objetivos:

- **Desempeño:**
¿Cómo implementar virtualización sin adicionar un sobrecosto excesivo (overhead)?
- **Control:**
¿Cómo ejecutar procesos de manera eficiente sin perder el control de la CPU?



Ejecución directa limitada (EDL)

Limited Direct Execution

Limited

Protección:

- Los procesos deben estar aislados (protegidos) entre sí
- El kernel del sistema operativo debe estar aislado de los procesos.
- Los dispositivos de hardware deben estar aislados de los procesos

Direct Execution

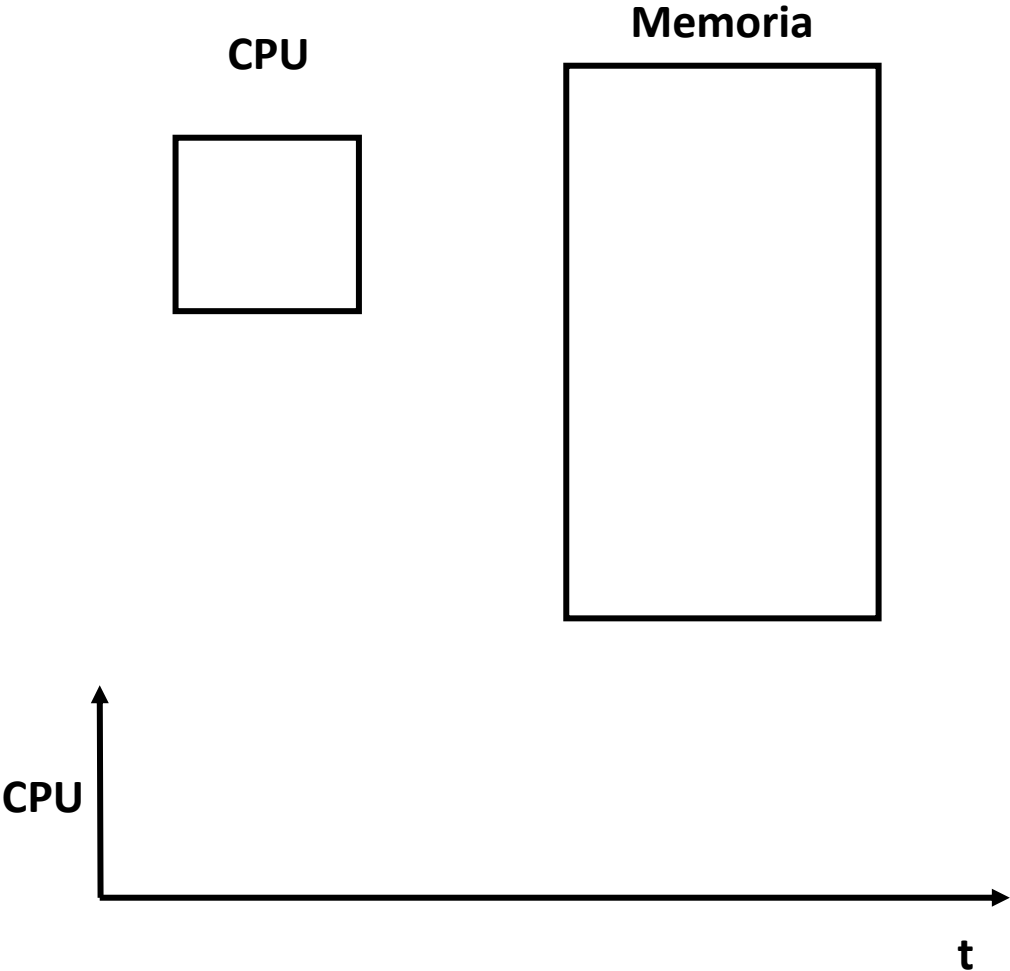
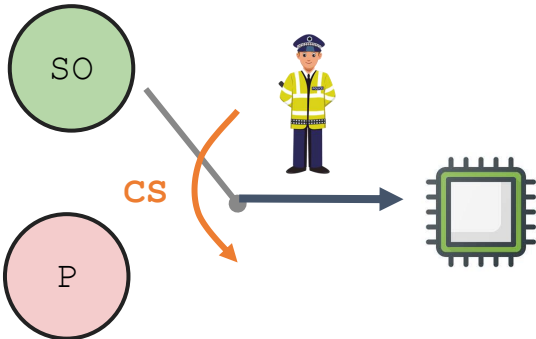
Desempeño:

- Las aplicaciones deben ejecutarse directamente en el procesador (El SO no debe intervenir y verificar la validez de cada una de las instrucciones que la aplicación desea ejecutar)

Ejecución directa limitada (EDL)

Ejecución directa:

El programa se ejecuta directamente en la CPU



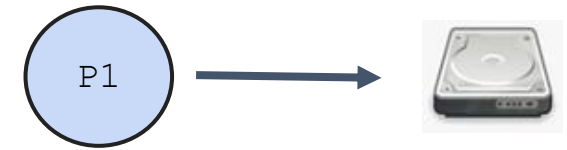
Sistema Operativo	Proceso de usuario
1. Crea una entrada en la lista de procesos 2. Asigna memoria al proceso 3. Carga el programa a memoria 4. Inicializa pila (<i>stack</i>) con <code>argc/argv</code> 5. Reinicia registros de CPU 6. Ejecuta llamada a <code>main()</code>	
	7. Ejecuta <code>main()</code> 8. Ejecuta <code>return</code> de <code>main()</code>
9. Libera la memoria del proceso 10. Elimina la entrada de la lista de procesos	

Ejecución directa limitada (EDL)

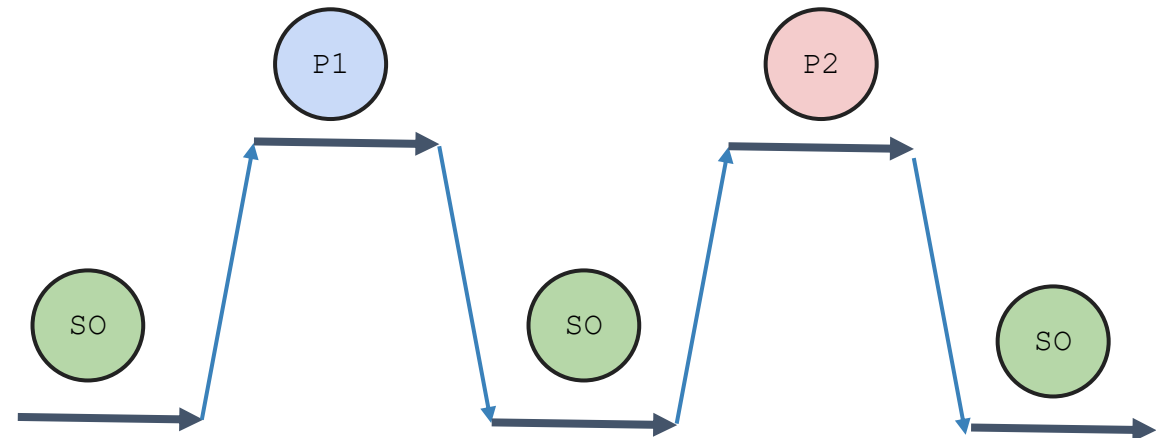
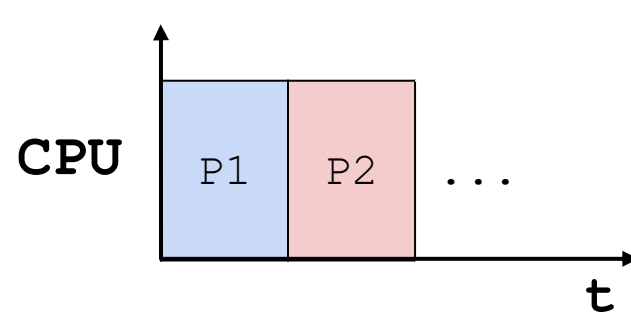
Problemas de la ejecución directa:

1. ¿Qué sucede si un proceso desea realizar una operación restringida?

- Emitir una solicitud E/S al disco.
- Obtener más recursos del sistema (memoria, CPU,...)
- Ejecutar operaciones especiales (Ejemplo: instrucción [LIDT](#)).



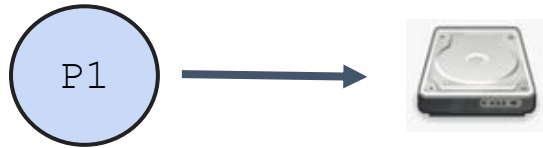
2. ¿Como hace el sistema Operativo para realizar el cambio de contexto?



Ejecución directa limitada (EDL)

Problemas de la ejecución directa:

1. ¿Qué sucede si un proceso desea realizar una operación restringida?

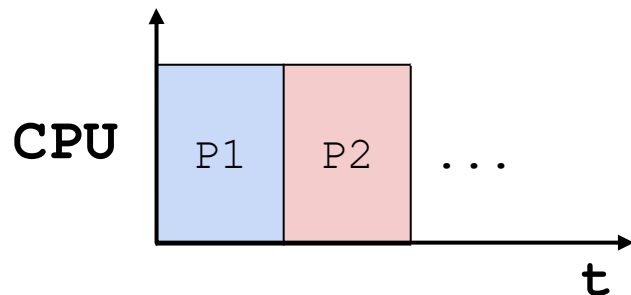


Limited

Direct

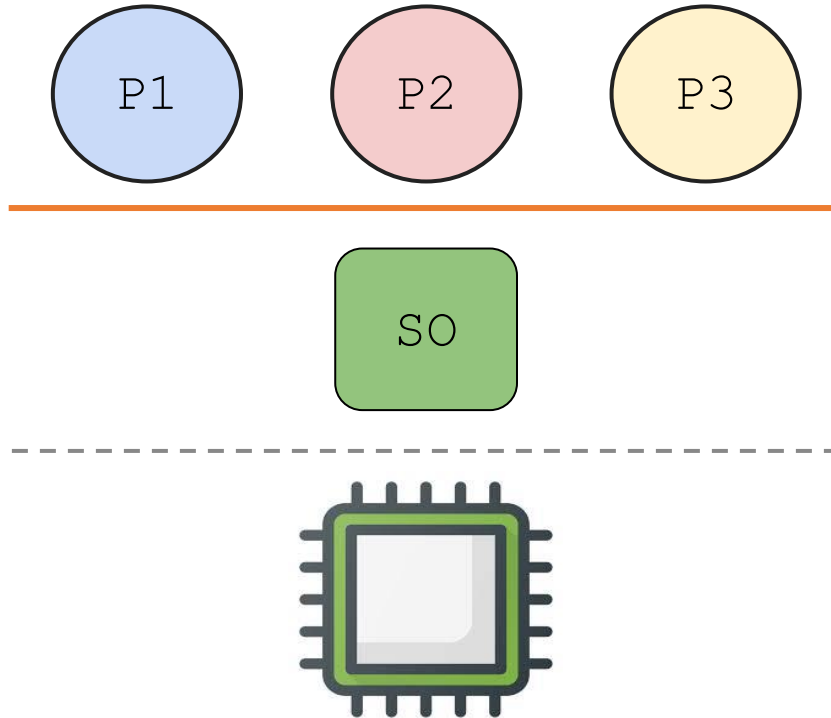
Execution

2. ¿Como hace el sistema Operativo para realizar el cambio de contexto?



Ejecución directa limitada (EDL)

Ejecución directa limitada

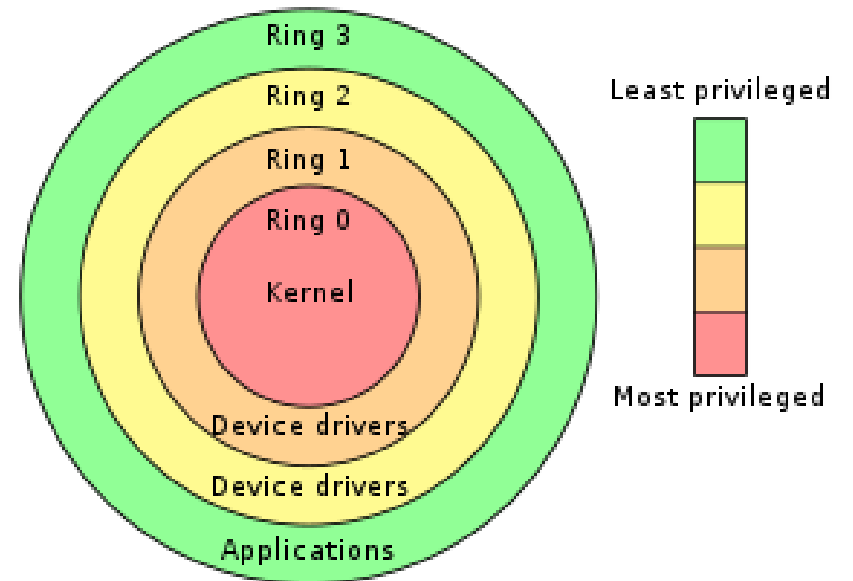
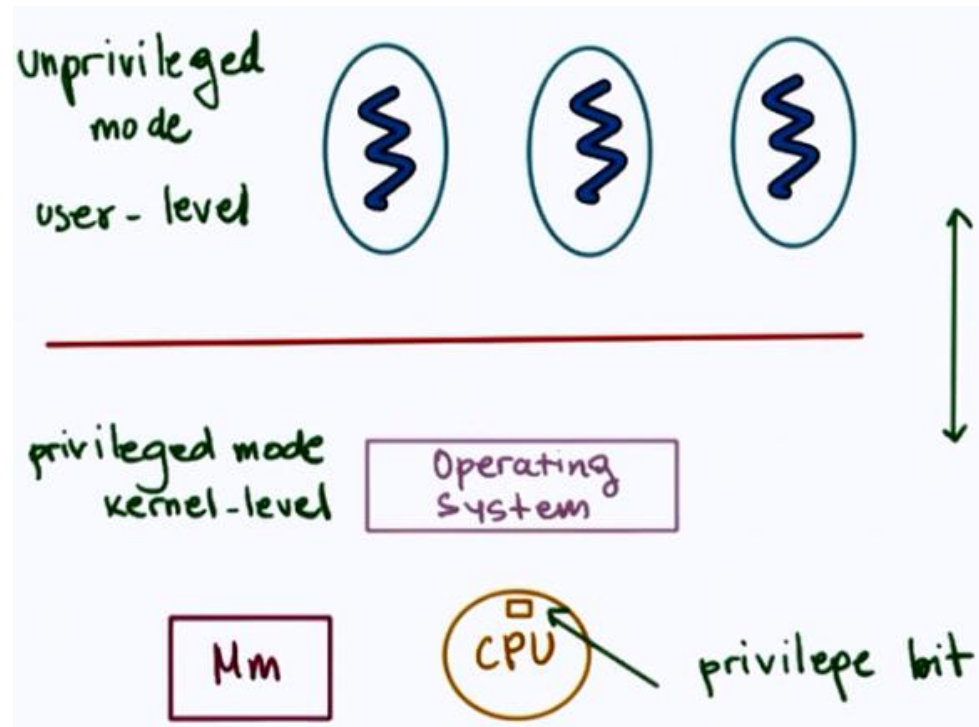


Ejecución directa limitada (EDL)

Problema 1: Operación restringida

Solución: usar transferencia de control protegida

- **Modo usuario:** Aplicaciones **no tienen** acceso total a los recursos de HW.
- **Modo kernel:** Sistema Operativo **tiene** acceso total a los recursos de HW



Ejecución directa limitada (EDL)

Modo kernel .vs. Modo usuario



Modo Usuario

- Se ejecutan las aplicaciones.
- Las aplicaciones solo pueden hacer un número limitado de cosas (No hay acceso privilegiado).



Modo Kernel

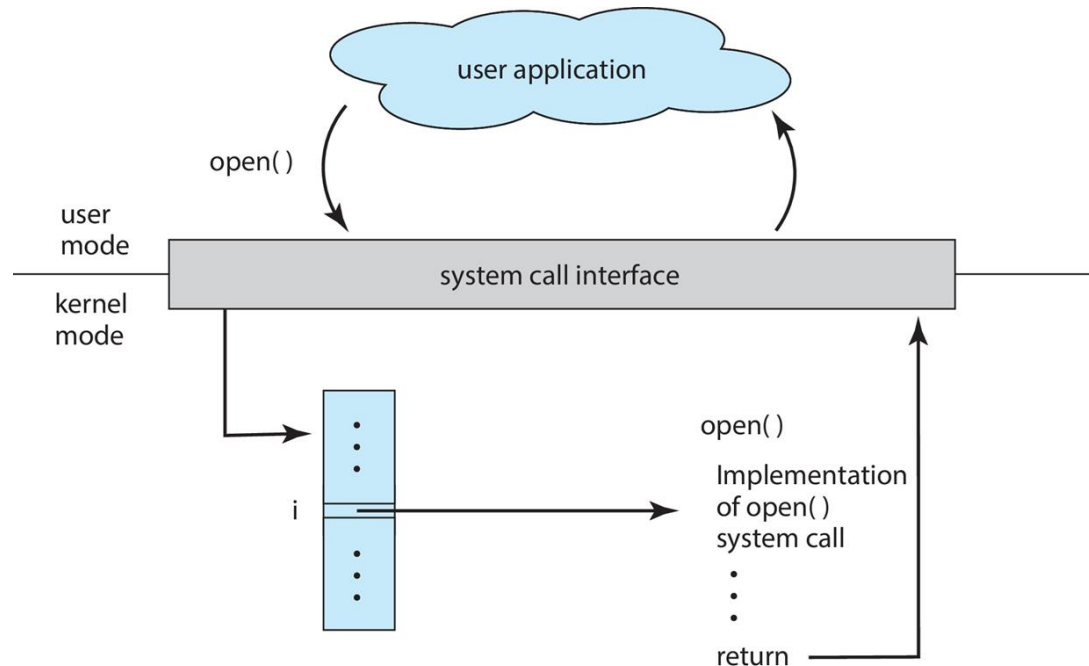
- Se ejecuta el sistema operativo
- El SO puede hacer lo que quiera (Acceso a instrucciones privilegiadas)
- Acceso al hardware.

Ejecución directa limitada (EDL)

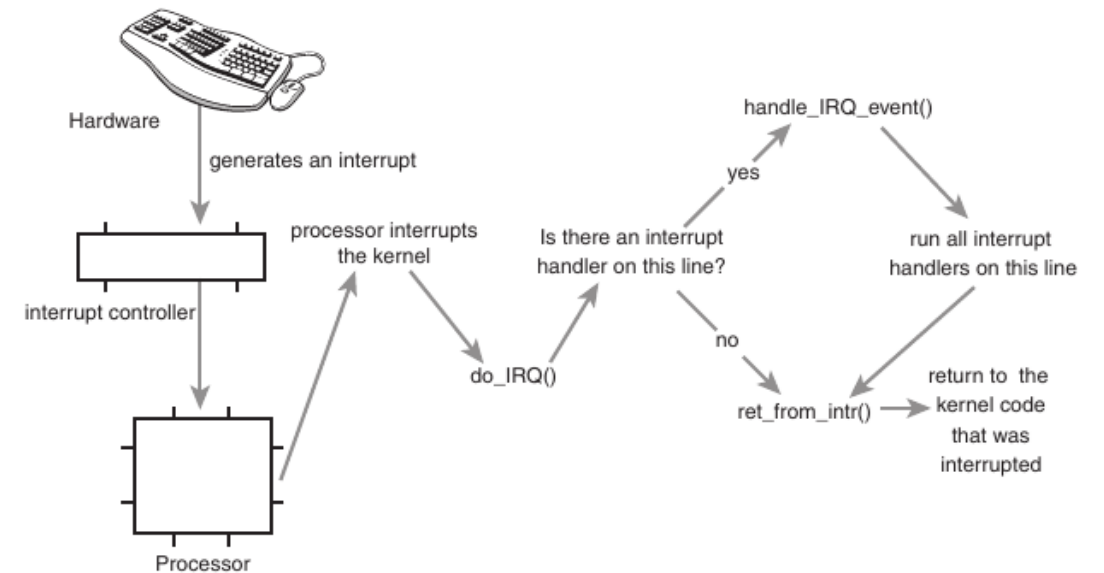
¿Cuándo y como se cambia entre los modos?

Se da cuando sucede un **evento**

Llamada al sistema (Syscall)

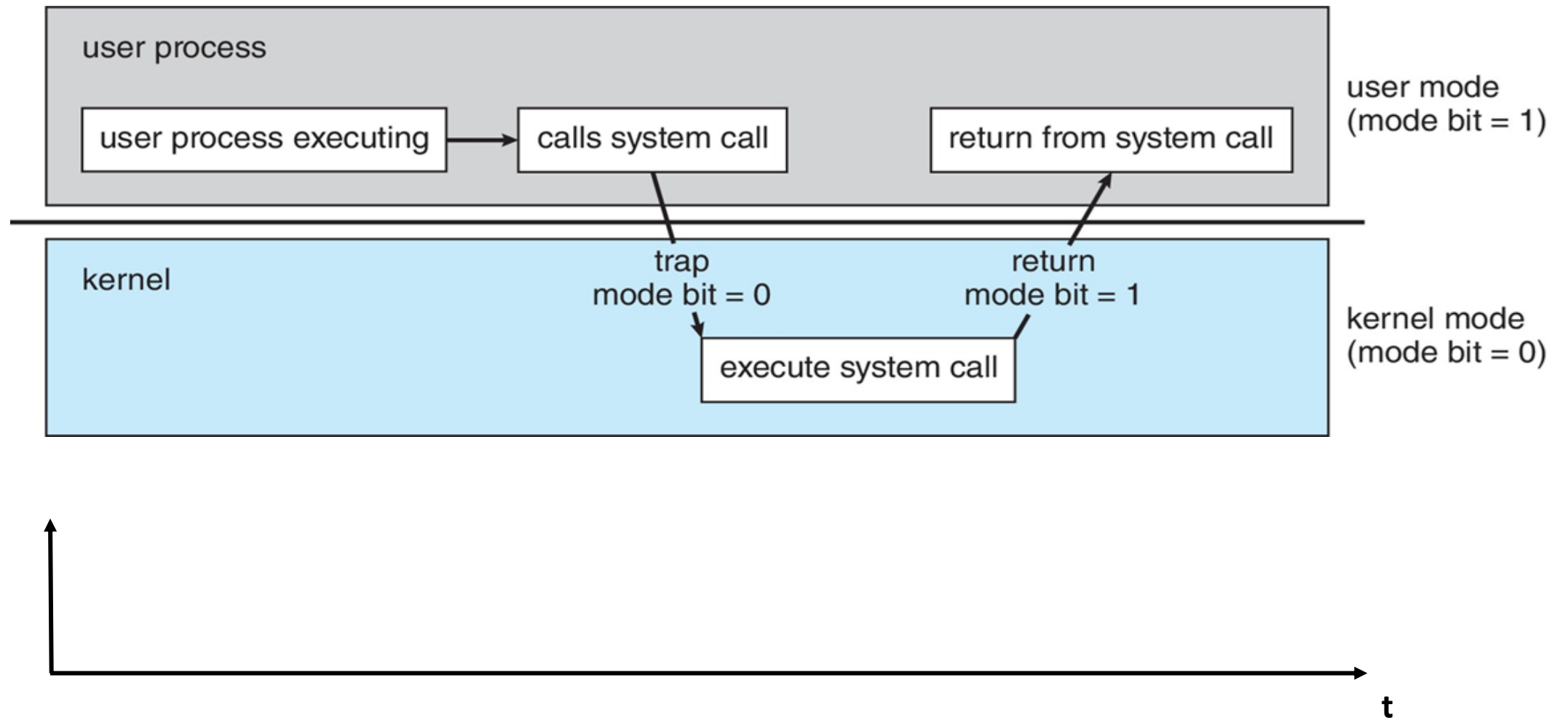


Interrupción



Ejecución directa limitada (EDL)

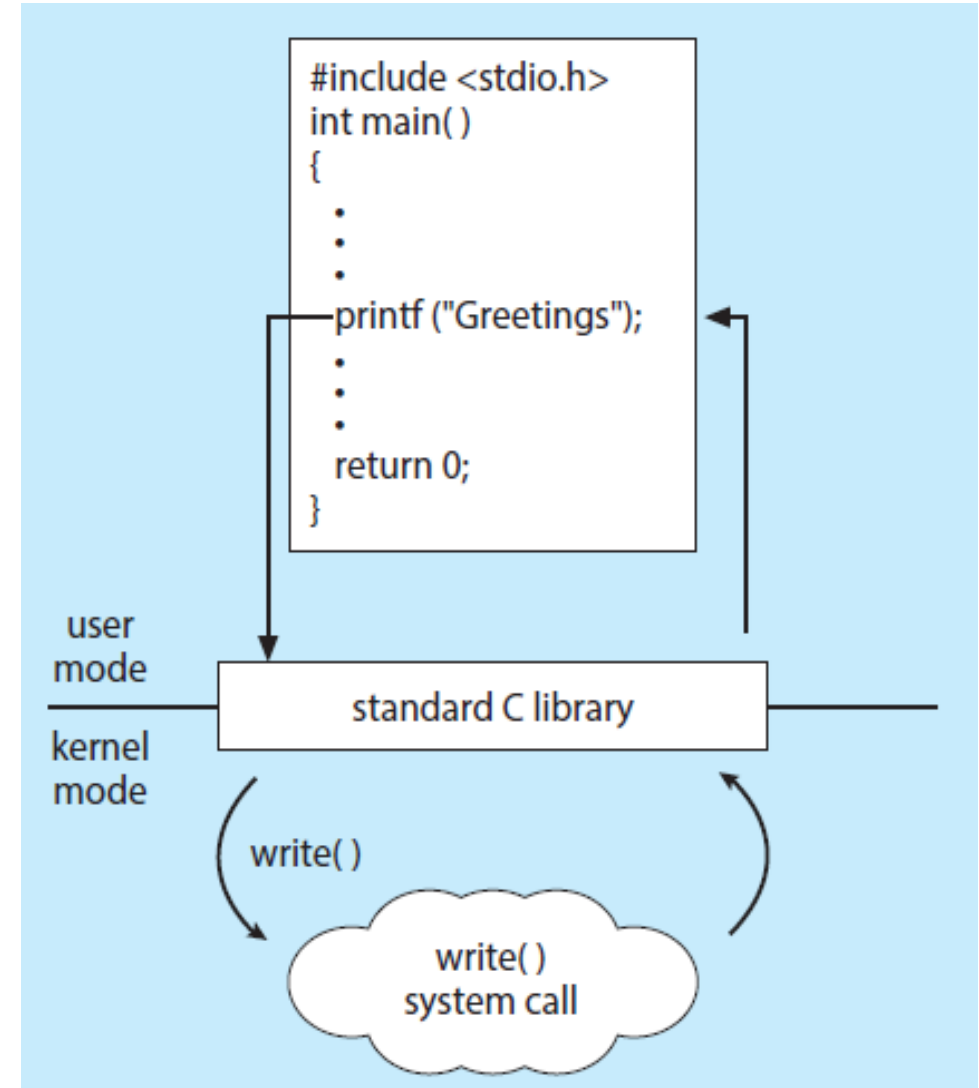
Transición entre modo usuario y modo kernel



Ejecución directa limitada (EDL)

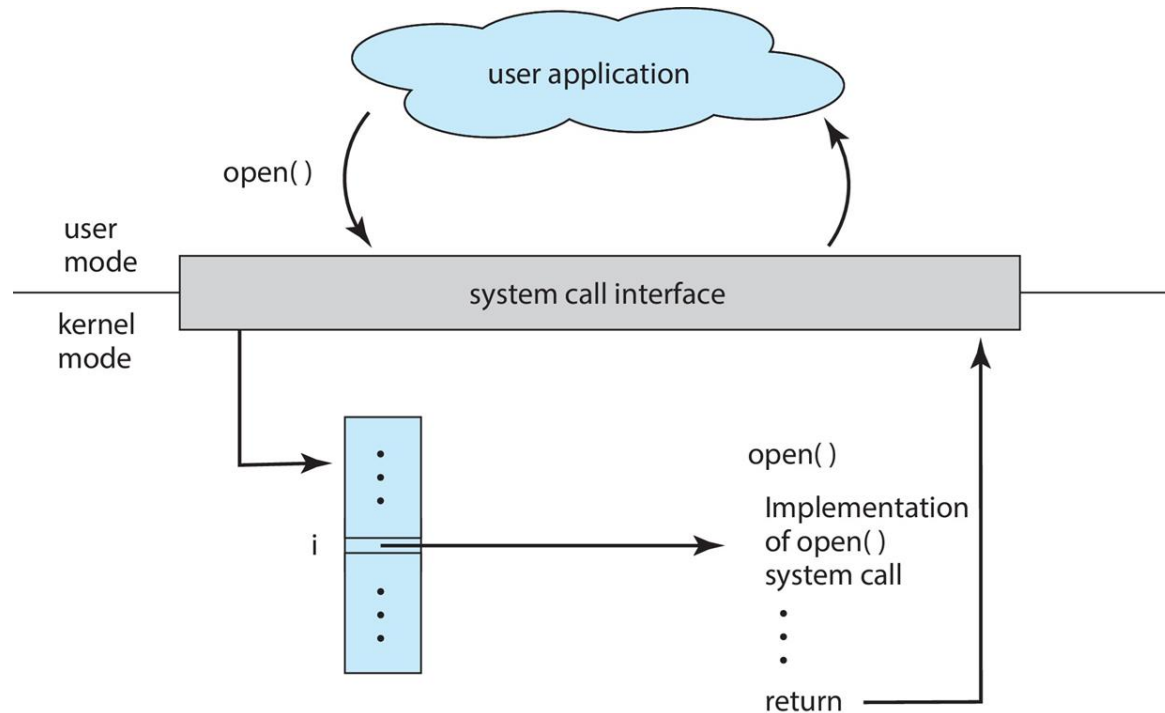
Llamadas al sistema (Syscalls)

- Interfaz de programación que permite al SO exponer ciertas funcionalidades clave a los programas de usuario:
- Normalmente escritas en un lenguaje de alto nivel (como C o C++).
- Los programas acceden a estas principalmente a través de una interfaz de programación (API) de alto nivel en lugar hacerlo directamente.
- APIs más comunes:
 - Win32 (Windows)
 - POSIX (Linux, Mac OS, Unix)
 - API de Java (JVM)



Ejecución directa limitada (EDL)

Llamadas al sistema (Syscalls)



EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

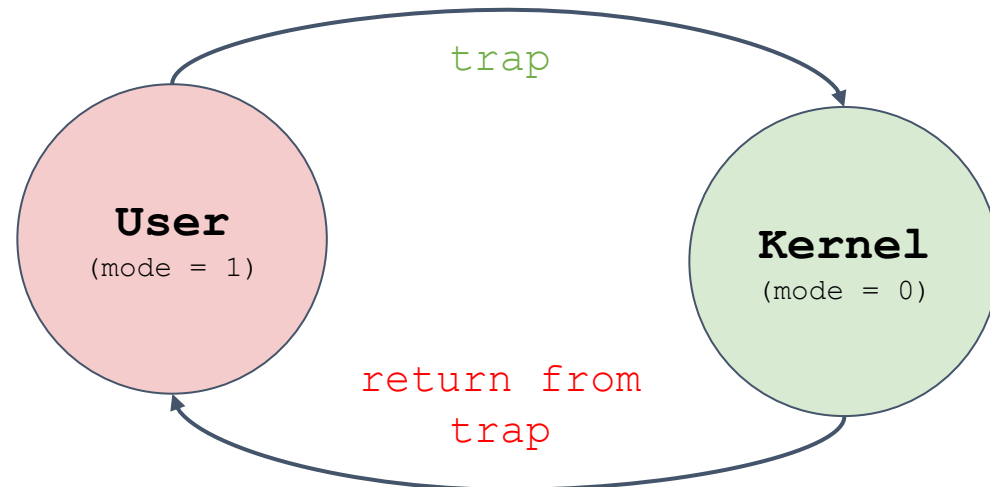
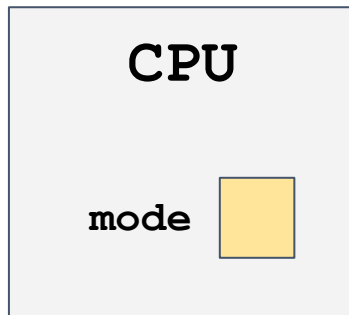
The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Ejecución directa limitada (EDL)

Llamadas al sistema (Syscalls)

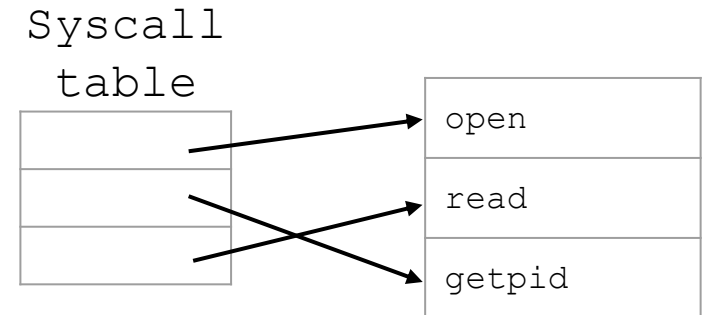
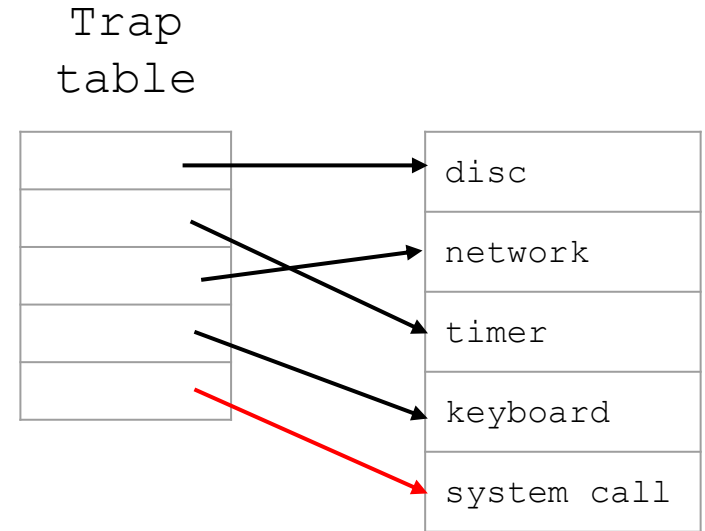
- **Instrucción `trap`**: Instrucción que permite simultáneamente:
 - Cambiar de modo usuario a modo kernel.
 - Cambiar el bit de modo de la CPU (mode bit = 0) para entrar al modo kernel.
- **Instrucción `return from trap`**: Permite:
 - Retornar al modo usuario.
 - Restablece CPU (mode bit = 1) a modo usuario.



Ejecución directa limitada (EDL) - Protocolo

SO @ Boot

SO @ Boot (Modo kernel)	Hardware
• Inicializa la trap table	
	• Almacena la dirección del Syscal handler • Ejecuta <code>return de main()</code>



Archivos relacionados: [trap.c](#), [trapasm.S](#), [vectors.pl](#) (Para mas información ver: A commentary on the sixth Edition Unix Operating System de J. Lions [\[link\]](#))

Recomendación: Antes de empezar esta parte vea el video **Computer Boot Process animation** [\[link\]](#)

Ejecución directa limitada (EDL) - Protocolo

SO @ Run

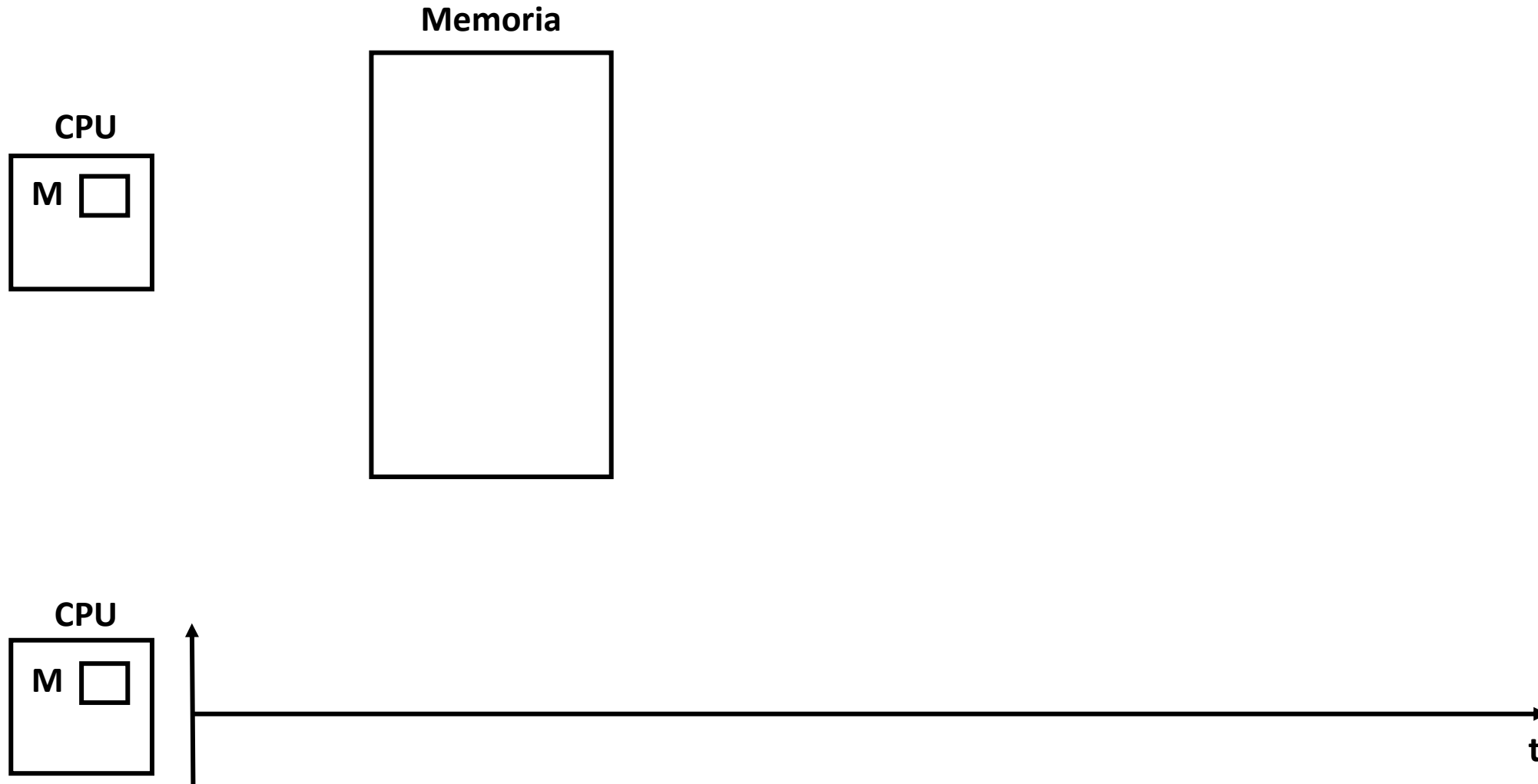
SO @ Run (Modo kernel)	Hardware	Programa (Modo usuario)
• Crea entrada en la lista de procesos • Asigna memoria al proceso • Carga el programa a memoria • Inicializa pila (stack) con <code>argc/argv</code> • Almacena valores de registros en el kernel stack • Return-from-trap		
	• Inicializa registros desde el kernel stack • Pasa a modo usuario • Salta a <code>main()</code>	
		• Ejecuta <code>main()</code> • ... • Llamado al sistema (trap)

Ejecución directa limitada (EDL) - Protocolo

SO @ Run (Modo kernel)	Hardware	Programa (Modo usuario)
	• Almacena valores de registros en el kernel stack • Pasa a modo kernel • Salta al trap handler	
• Atiende el evento <code>trap</code> • Realiza el trabajo solicitado por la llamada al sistema • <code>Return-from-trap</code>		
	• Restaura valores de registros desde el kernel stack • Pasa a modo usuario • Salta a PC después de <code>trap</code>	
		• ... • <code>return</code> de <code>main()</code> • Llamado al sistema <code>exit (trap)</code>
• Libera la memoria del proceso • Elimina la entrada de la lista de procesos.		

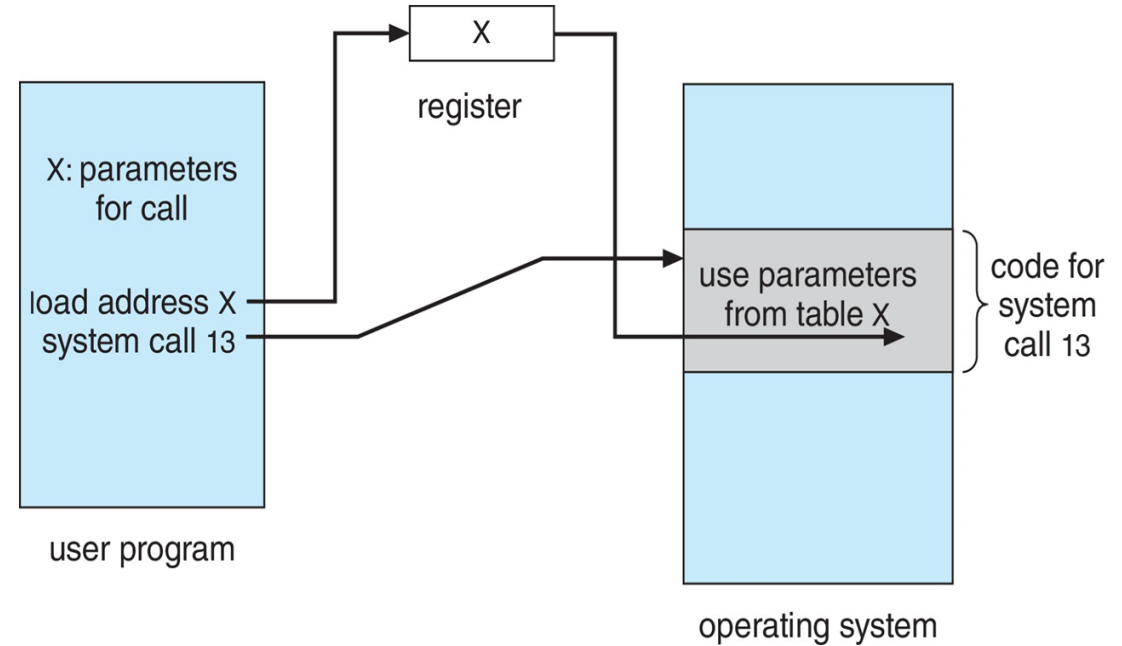
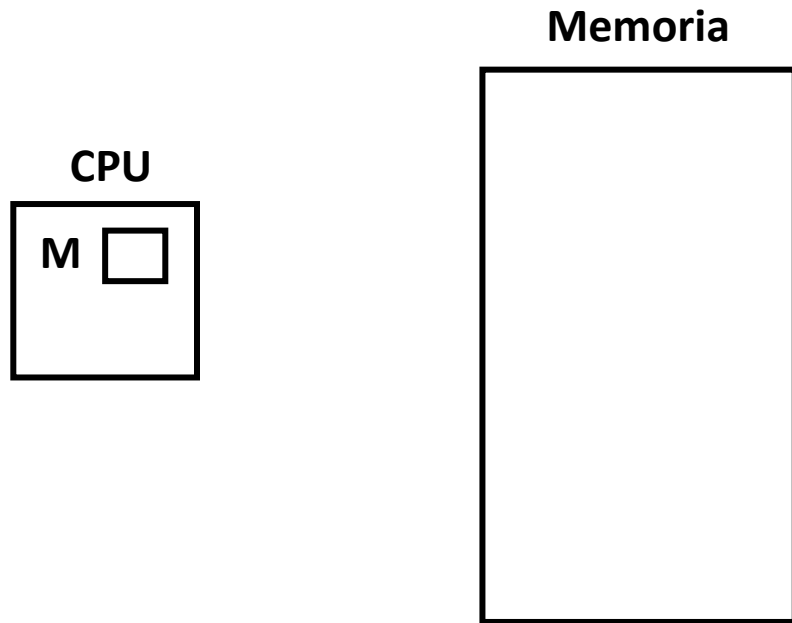
Ejecución directa limitada (EDL)

Ejecución directa limitada - Ejemplo



Ejecución directa limitada (EDL)

Ejecución directa limitada - Ejemplo



Ejecución directa limitada (EDL)

Ejecución directa limitada - Resumen

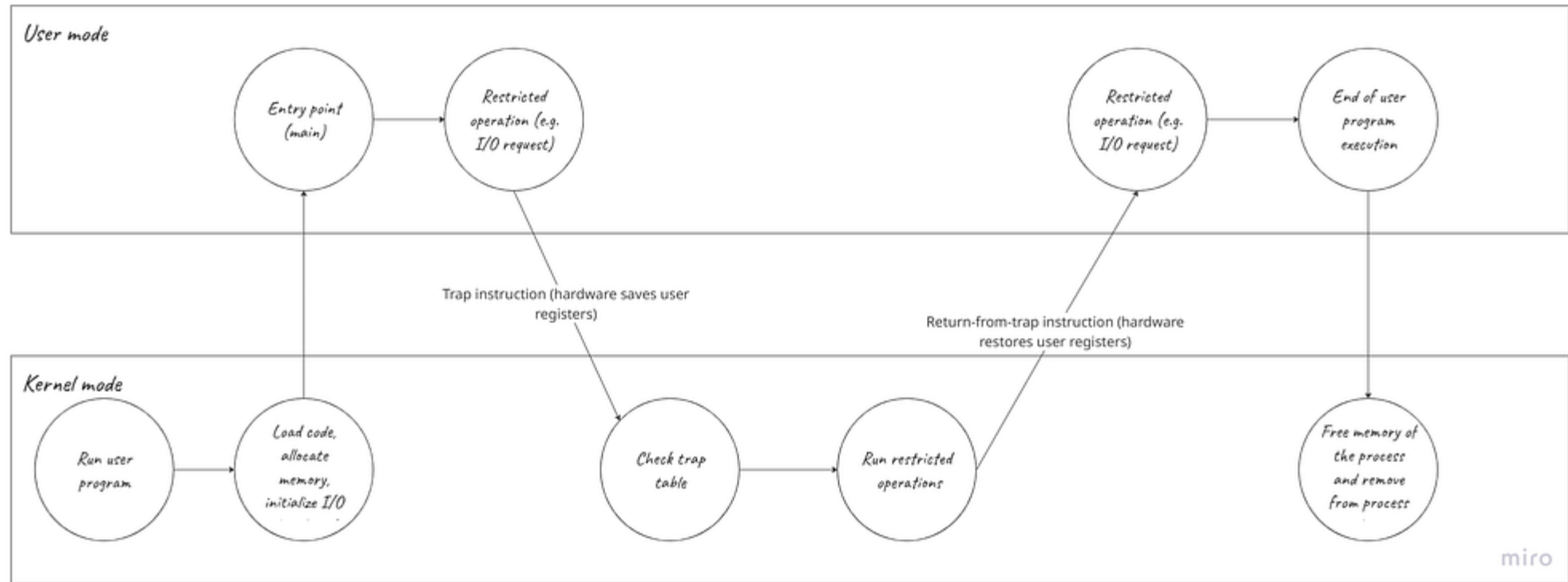
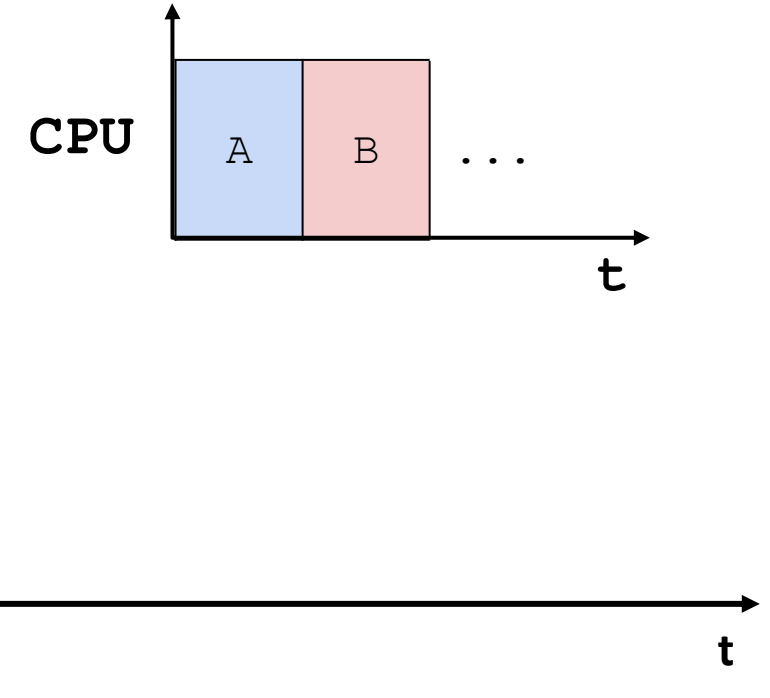
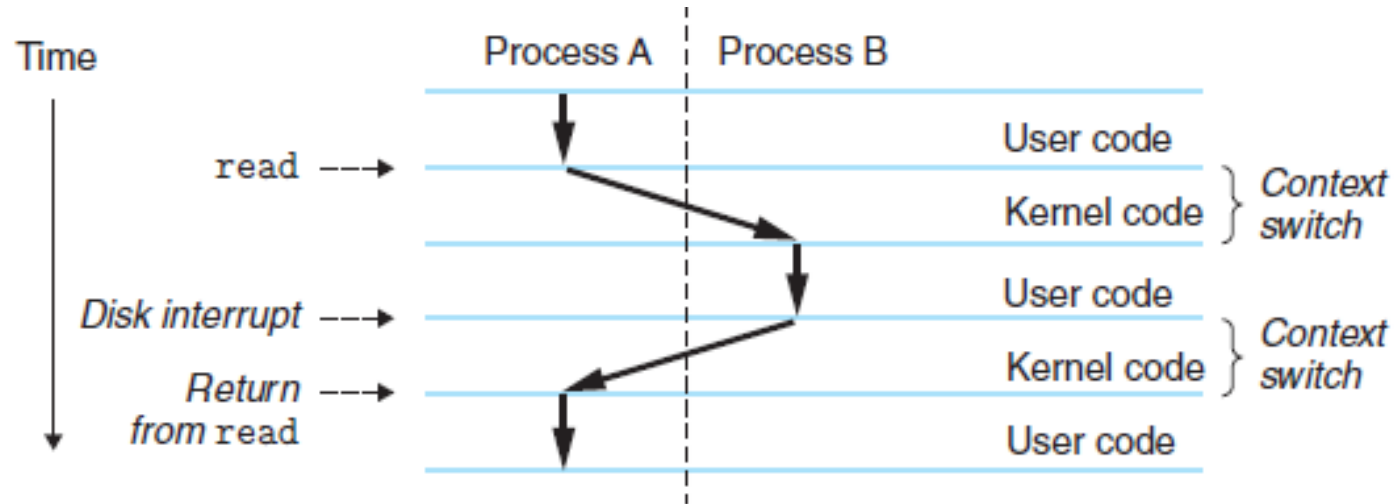


Imagen tomada del siguiente [link](#)

Ejecución directa limitada (EDL)

Problema 2: Cambio entre procesos



¿Cómo hace el SO para retomar el control de la CPU y realizar cambios entre procesos?

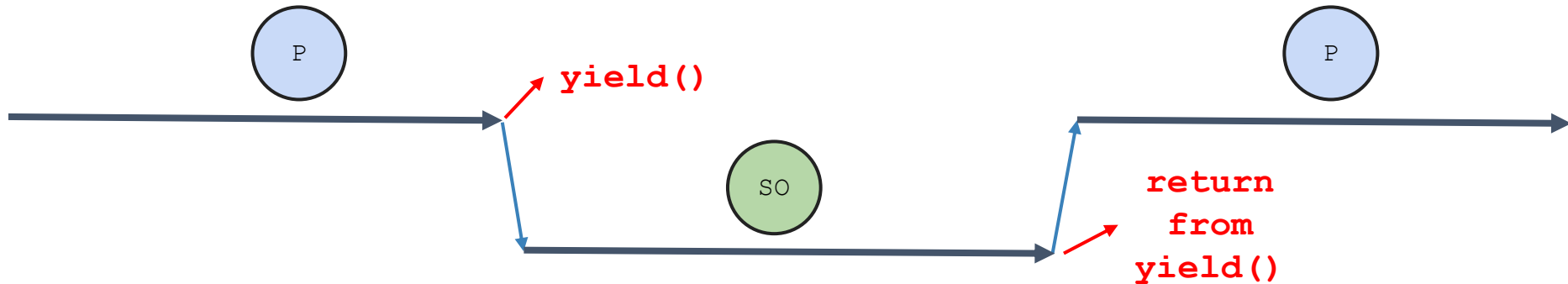
Hay dos formas:

1. **Propuesta cooperativa:** Se cede el control mediante llamados a sistema.
2. **Propuesta no-cooperativa:** El SO toma el control.

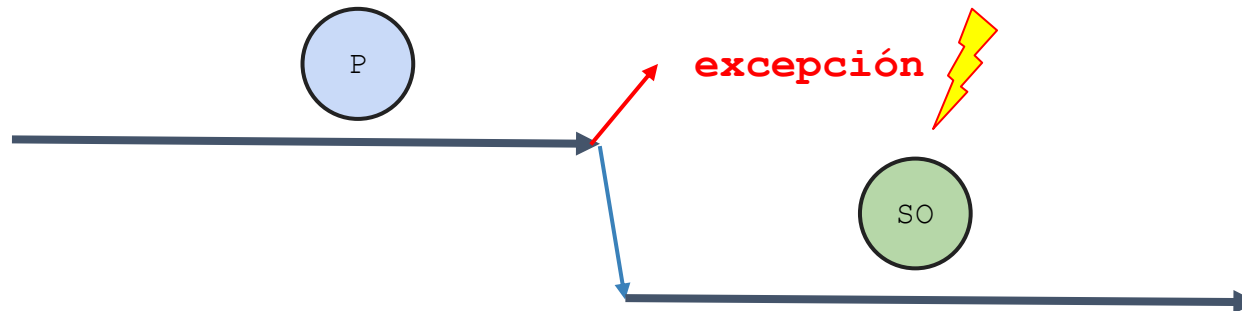
Ejecución directa limitada (EDL)

Propuesta cooperativa (no apropiativa)

- Los procesos liberan la CPU periódicamente
- El cambio de contexto se da empleando llamadas a sistema.
 - **Empleo de la llamada a sistema.**



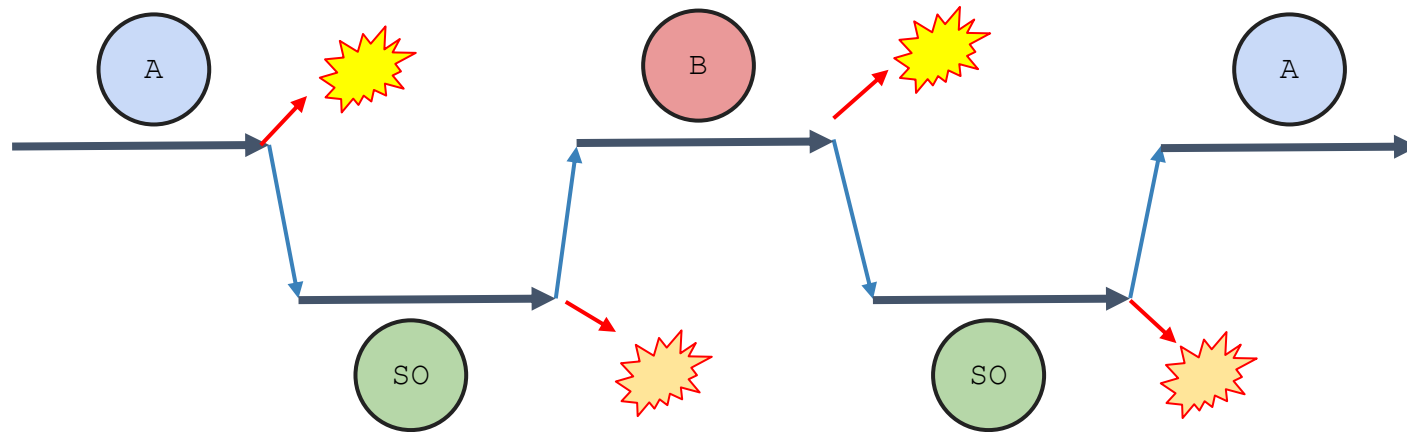
- **Operación ilegal:** División por cero, acceso ilegal a memoria, etc.



Ejecución directa limitada (EDL)

Propuesta cooperativa (no apropiativa)

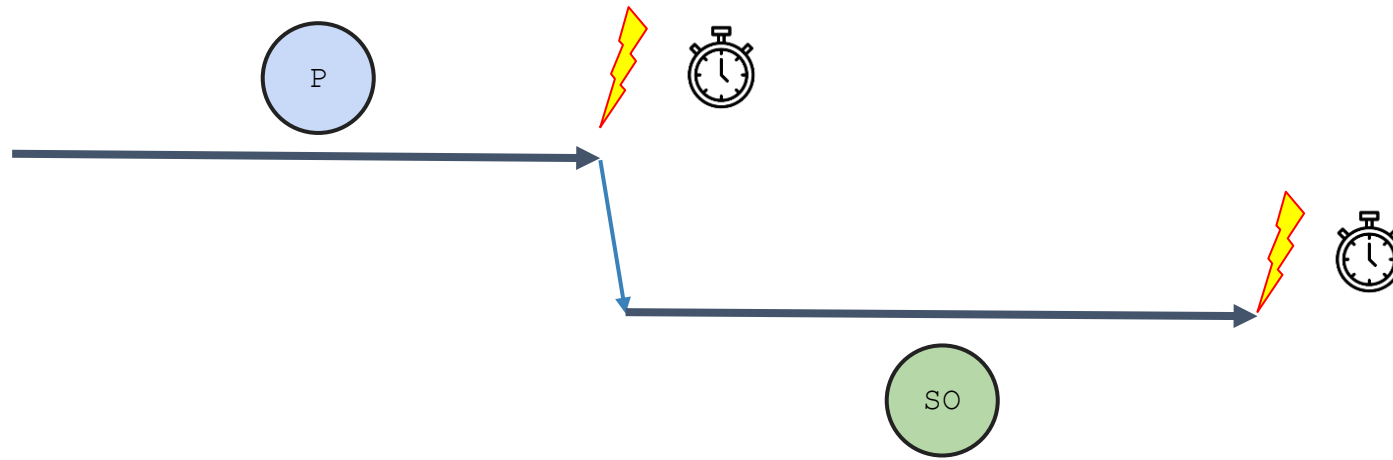
- Cuando el proceso libera la CPU, es el sistema operativo quien decide el próximo proceso que la usara (Como se hace lo veremos luego).



Ejecución directa limitada (EDL)

Propuesta no cooperativa (apropiativa)

- El sistema operativo es quien toma el control de la CPU mediante interrupciones por timer.
- El timer lanza interrupciones cada cierto tiempo (algunos milisegundos).

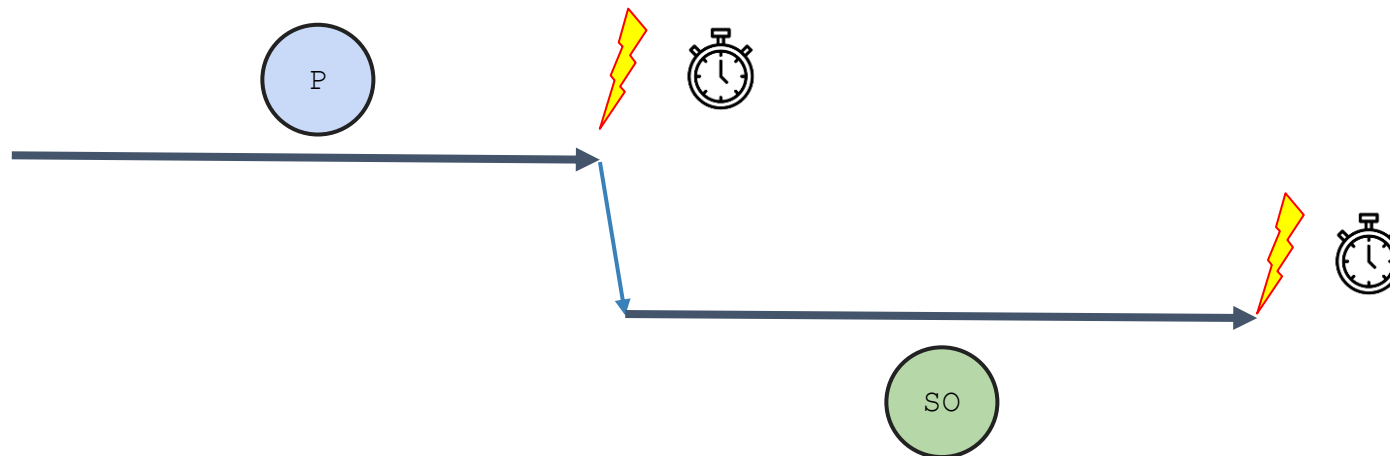
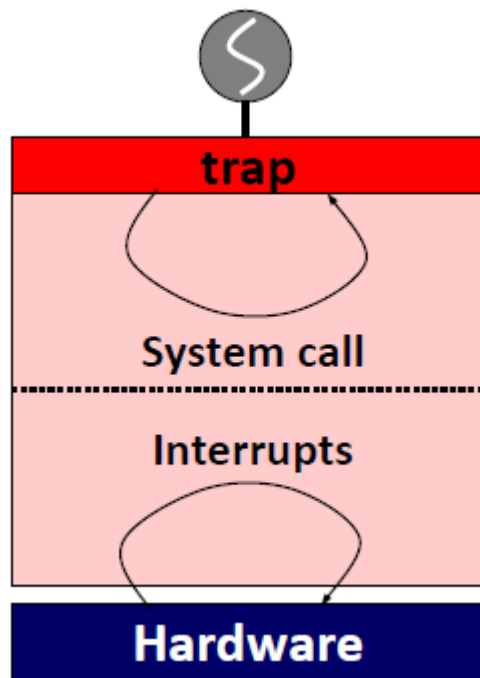


- Cuando se lanza una ejecución por timer:
 1. El proceso en ejecución es **suspendido**.
 2. Se almacena el estado del proceso.
 3. Una **rutina de atención** a la interrupción es ejecutada

Ejecución directa limitada (EDL)

Propuesta no cooperativa (apropiativa)

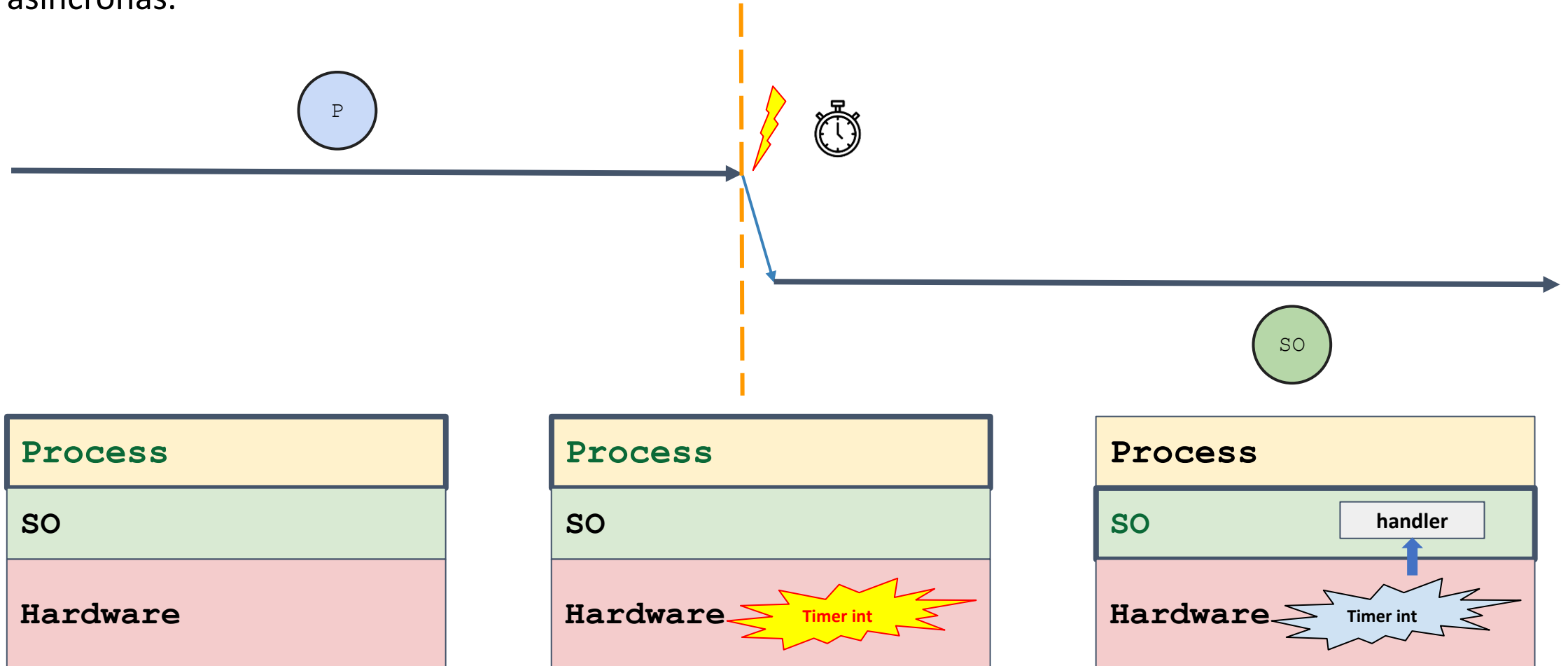
- La interrupción del timer le da a la SO la habilidad de **retomar** el control de la CPU.



Ejecución directa limitada (EDL)

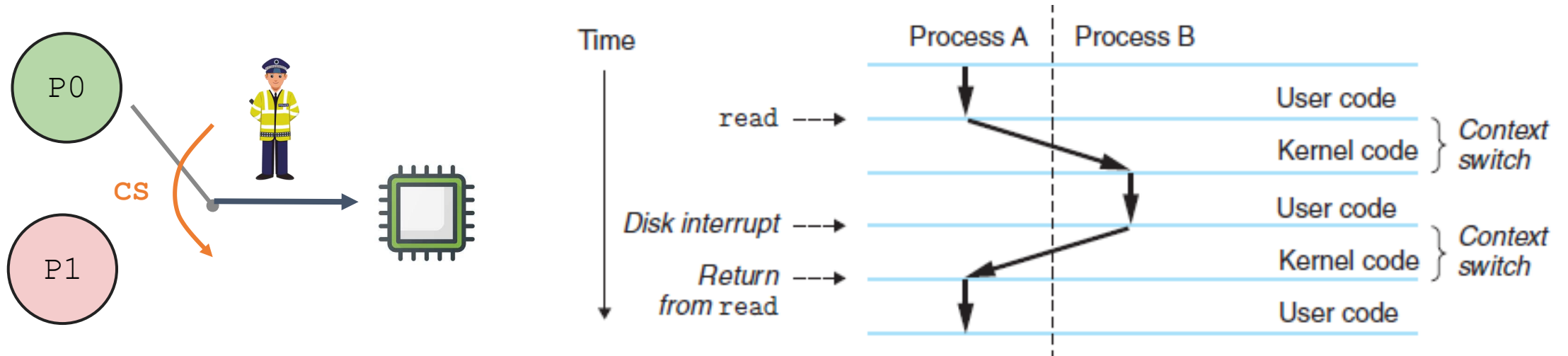
Sobre las interrupciones

- Las interrupciones son generadas por hardware.
- Son asíncronas.



Interrupción y varios procesos

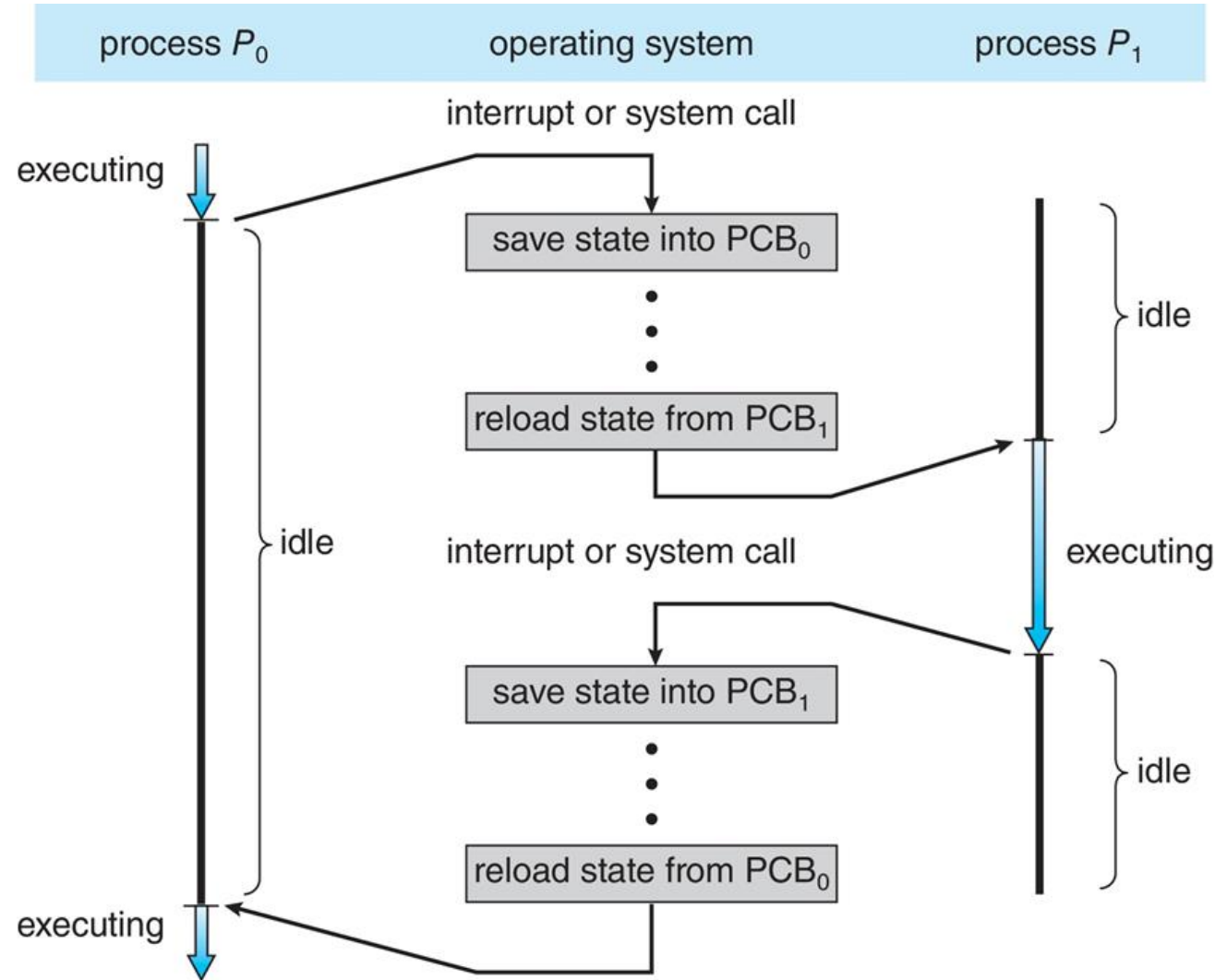
- Cuando hay una interrupción el scheduler tiene que tomar una decisión sobre el paso a seguir:
 1. Continuar con el proceso actual.
 2. Cambiar a un proceso diferente.
- Si la decisión es cambiar, el SO ejecuta un cambio de contexto (**context switch**).



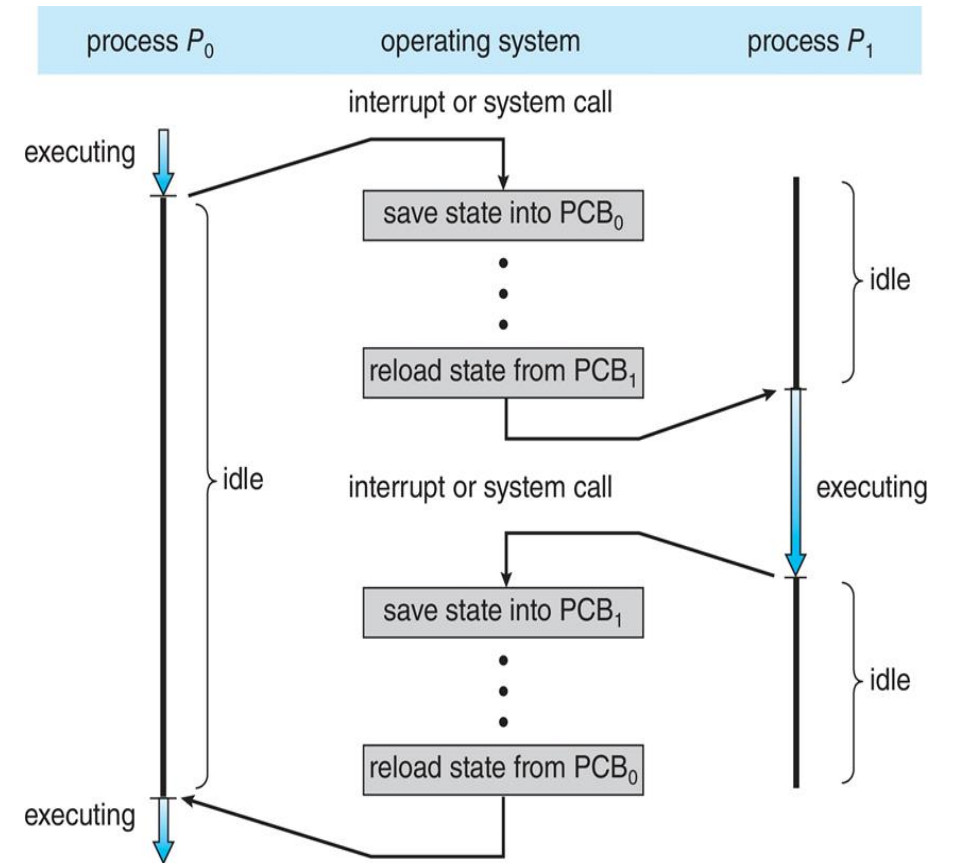
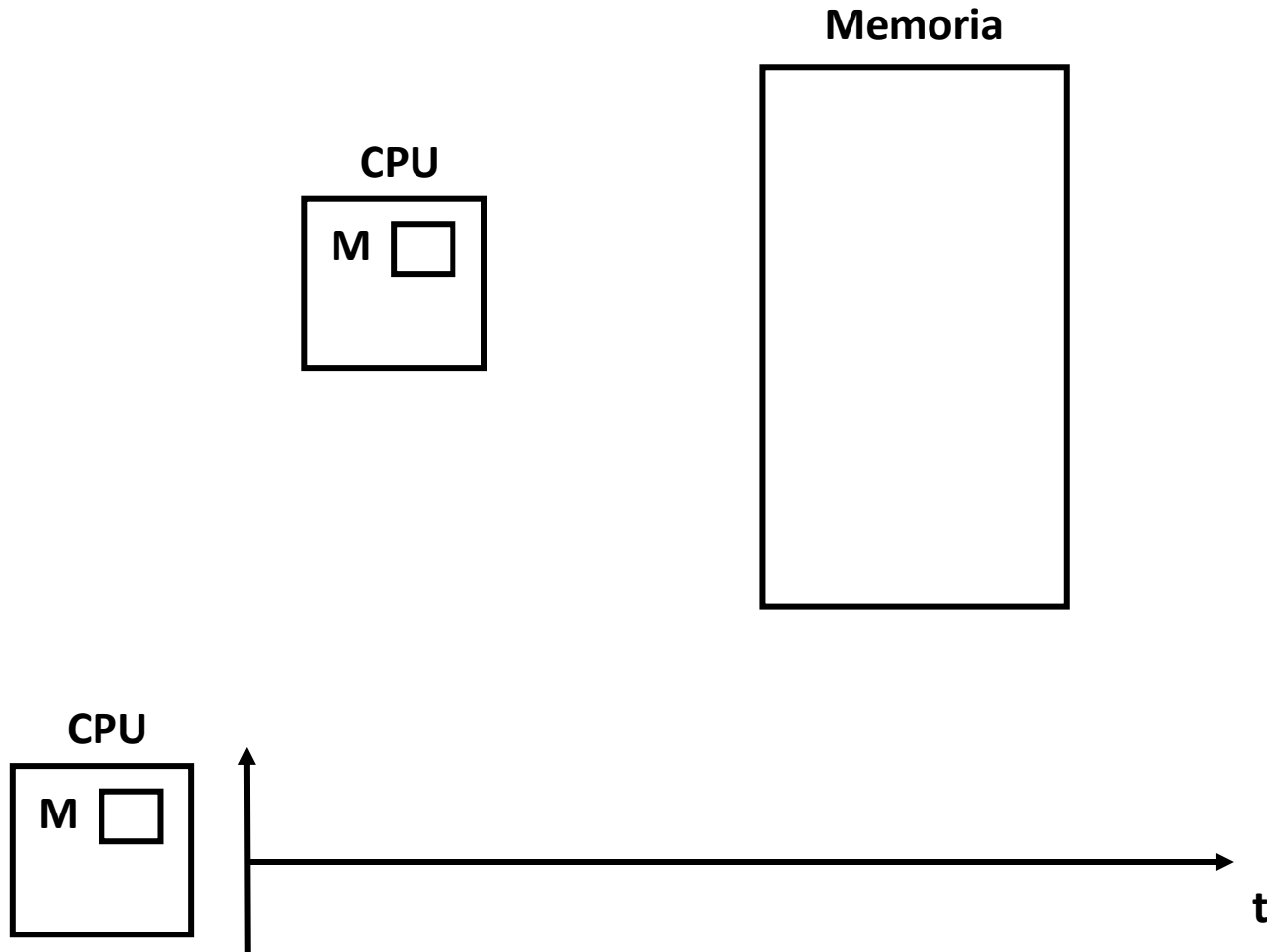
Cambio de contexto

Cuando hay un cambio de contexto pasa lo siguiente:

- Se almacena el estado del proceso en ejecución en el PCB, esto implica almacenar los valores de la CPU en el **kernel stack**:
 1. Registros de propósito general.
 2. PC.
 3. kernel Stack pointer.
- Se restauran los valores del proceso que se va a ejecutar desde el **kernel stack**.
- Se cambia al **kernel stack** del proceso que se va a ejecutar.



Cambio de contexto



Cambio de contexto

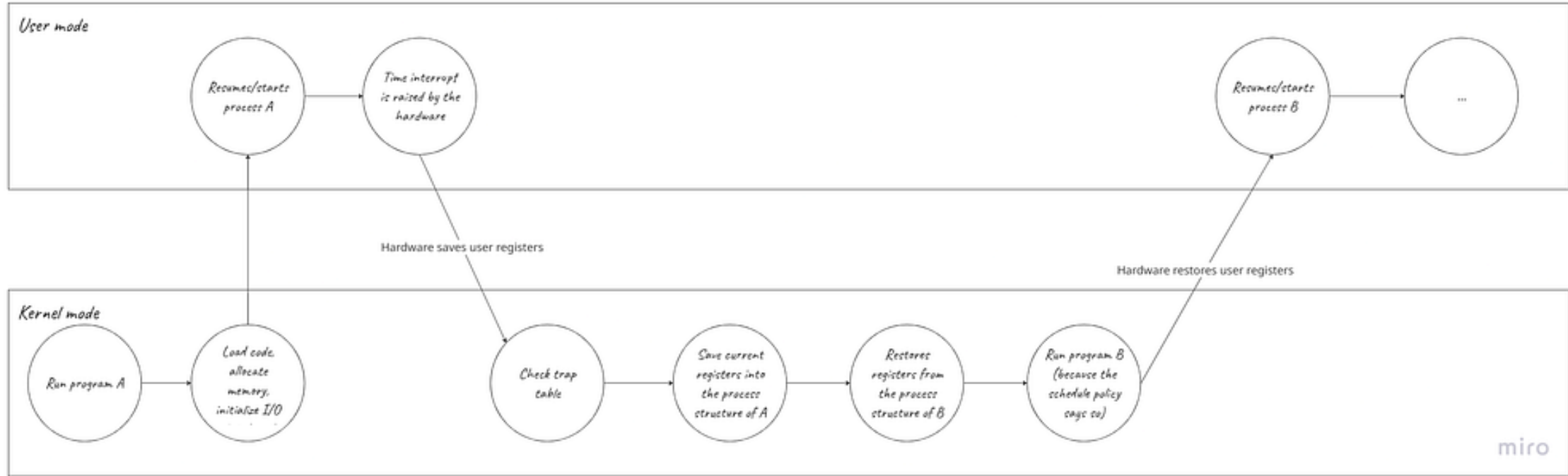


Imagen tomada del siguiente [link](#)

Ejecución directa limitada (timer)

SO @ Boot

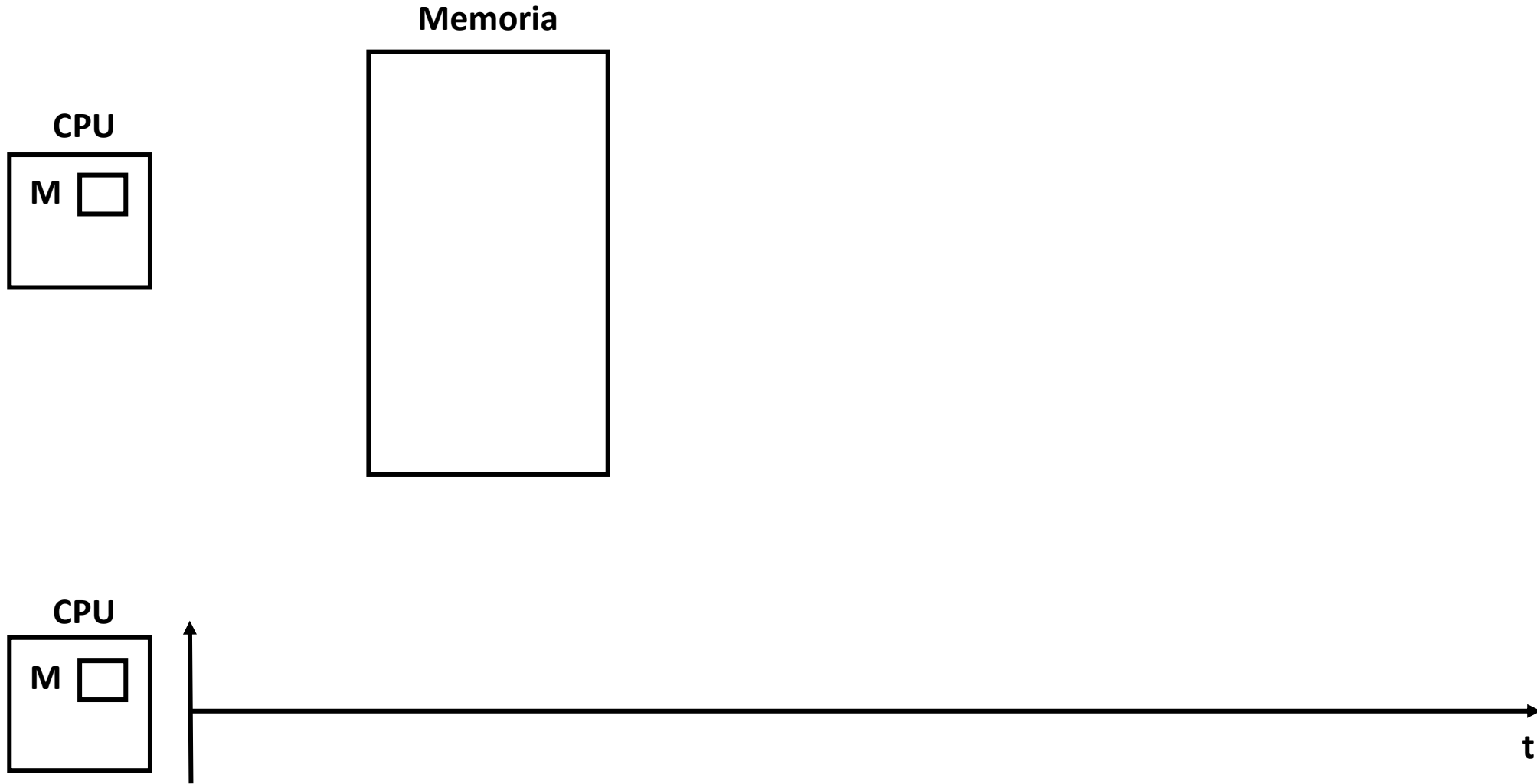
SO @ Boot (Modo kernel)	Hardware
• Inicializa la trap table	
	• Almacena la dirección del Syscal handler y el timer handler
• Inicia el timer interrupt	
	• Inicia el timer • Interrumpe la CPU en X ms

Ejecución directa limitada (timer)

SO @ Run

SO @ Run (Modo kernel)	Hardware	Programa (Modo usuario)
		• Proceso A • ...
	• Interrupción del timer • Almacena registros de A en el kernel stack de A • Pasa a modo kernel • Salta al timer handler	
• Atiende el trap • Llama la rutina switch() ◦ Almacena registros de A a PCB de A ◦ Restaura registros de B desde PCB de B ◦ Cambia a kernel stack de B • Return-from-trap (hacia B)		
	• Restaura registros de B a kernel stack de B • Pasa a modo usuario • Salta a PC de B	
		• Proceso B • ...

Múltiples interrupciones

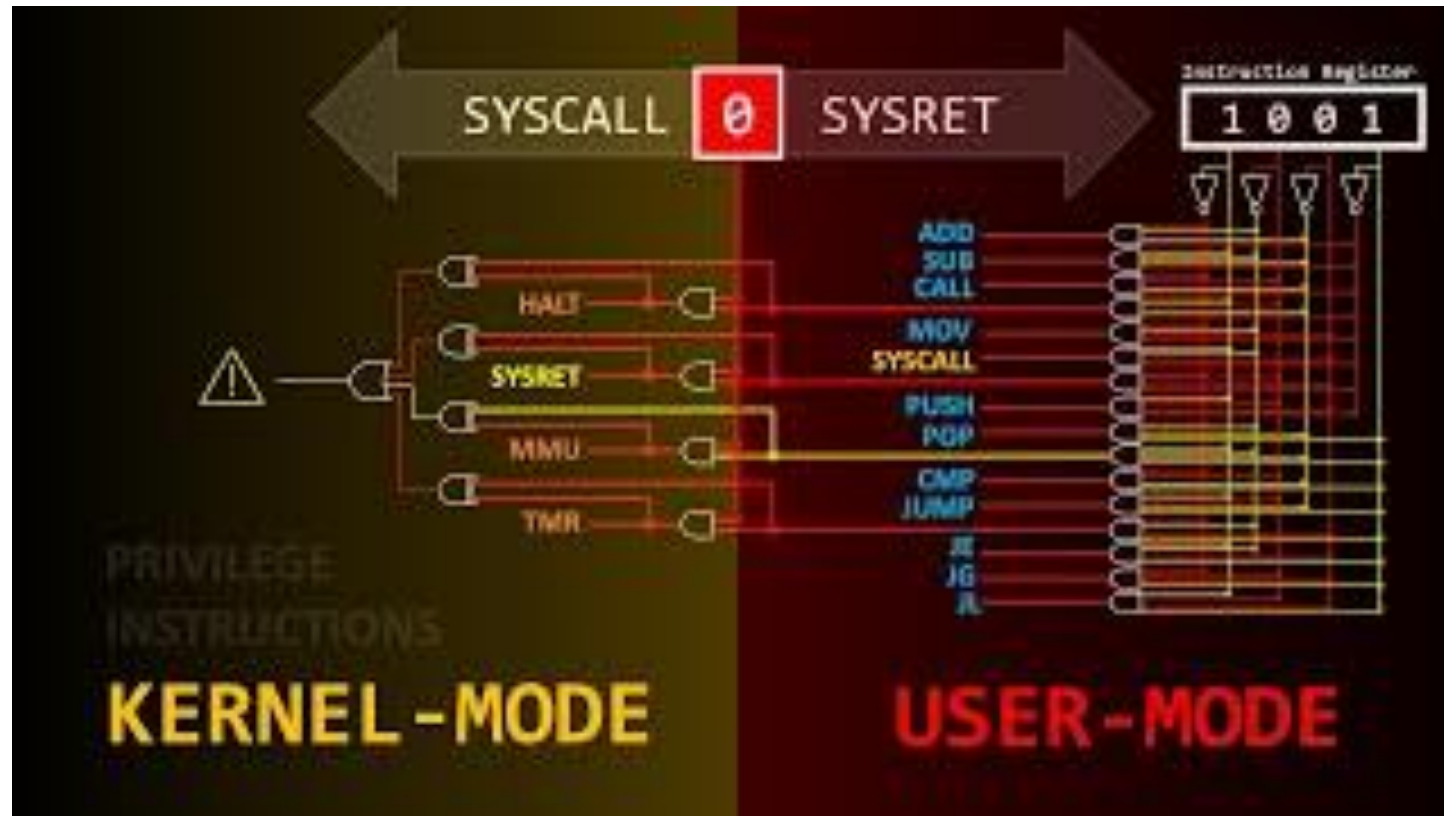


Múltiples interrupciones

- ¿Qué pasa si, durante una interrupción o una rutina de atención (**trap handler**), llega otra interrupción?
- El SO se encarga de estas situaciones:
 - **Deshabilita interrupciones** durante la atención a las interrupciones.
 - Utiliza esquemas de protección (bloqueo) avanzados para **proteger** el acceso concurrente a **estructuras de datos internas**.

Para comprender todo

Ahora que ha finalizado la clase, se recomienda que vea el video **How a Single Bit Inside Your Processor Shields Your Operating System's Integrity** ([link](#)) para que refuerce los conceptos anteriormente explicados.



Referencias

- **Operating Systems Three Easy Pieces** ([website](#): Capítulos [1](#) y [2](#))
- Videos de youtube: Clase 1 - Introducción a los sistemas operativos ([link](#)).
- Página de Remzi H. Arpaci-Dusseau ([link](#))
- Operating System Concepts - Tenth Edition ([website](#))
- Operating Systems Notes ([link](#))
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/1-Overview.md>
- <https://myaut.github.io/dtrace-stap-book/kernel/proc.html>
- <https://myaut.github.io/dtrace-stap-book/index.html>
- <https://www.technologyuk.net/computing/computer-software/operating-systems/operating-system-utilities.shtml>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/simple-threads/>
- <https://courses.cs.duke.edu/spring19/compsci590.1/slides/>
- <https://w3.cs.jmu.edu/kirkpams/OpenCSF/Books/csf/html/index.html>
- <https://github.com/cfenollosa/os-tutorial>